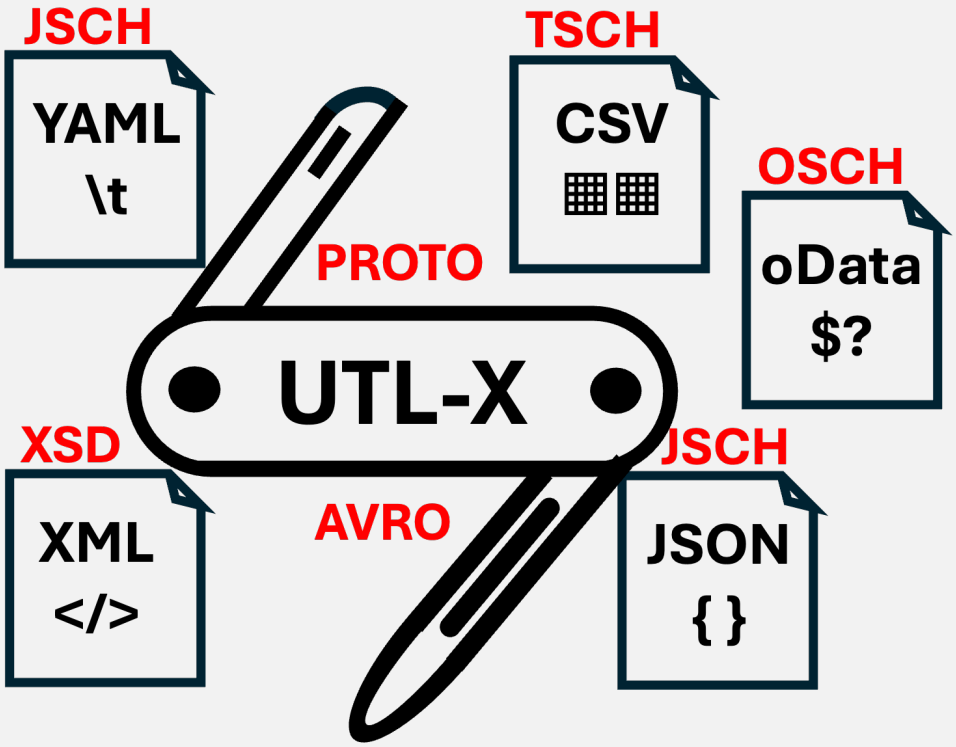
GLOMIDCO

UTLXe

Production Transformation Engine



Deployment and Operations Guide

Running UTLXe on Azure Container Apps — From First Deploy to Production

Ir. Marcel A. Grauwen

First Edition — 2026

UTLXe on Azure — Deployment and Operations Guide

Published by GLOMIDCO B.V., The Netherlands

First edition, 2026.

UTL-X is open source software. The language specification, CLI tool, and standard library are freely available at `https://github.com/grauwen/utl-x`.

This is a companion guide to *UTL-X: One Language, All Formats* (ISBN 978-90-819728-1-9), which covers the complete language specification, standard library, and format system.

Typeset with Typst in New Computer Modern.

Table of Contents

1 What UTLXe Does for Your Business 7
1.1 The Problem 7
1.2 What UTLXe Is 7
1.3 Who Uses UTLXe 7
1.4 Compared to Alternatives 8
1.5 Azure Marketplace: What You Get 8
1.6 Business Scenarios 9
1.7 Pricing Model 9
1.8 What This Book Covers 9
2 Why UTLXe? 10
2.1 The Transformation Problem 10
2.2 What the UTLXe Azure Offering Does 11
2.3 The Cost of Embedded Transformation 25
2.4 How UTLXe Runs on Azure 26
2.5 The Deployment Workflow 27
2.6 What UTLXe Does Not Do 27
2.7 When to Use UTLXe 27
2.8 Migrating from BizTalk Server 27
3 Quick Start 29
3.1 What You Get 29
3.2 Deploy from the Azure Marketplace 29
3.3 Set the Admin Key 29
3.4 Verify the Deployment 29
3.5 Write Your First Transformation 30
3.6 Upload It 30
3.7 Test It 30
3.8 Send a Real Message 31
3.9 What Just Happened 31
3.10 Next Steps 31
4 Writing Transformations 32
4.1 The .utlx File Structure 32
4.2 Property Access 32
4.3 Object Construction 32
4.4 Let Bindings 33
4.5 Conditionals 33
4.6 Array Operations 33
4.7 Common Standard Library Functions 34
4.8 Multi-Format Transformations 35
4.9 User-Defined Functions 35
4.10 Worked Example: Invoice Transformation 36
4.11 Where to Learn More 37
5 The Admin API 38
5.1 Architecture: Two Ports, Two Purposes 38
5.2 The Admin Web UI (Optional) 38
5.3 Authentication 38
5.4 Uploading Transformations 39
5.5 Managing Schemas 40
5.6 Testing Before Go-Live 40
5.7 Listing and Inspecting 41

5.8 Updating Transformations ... 42
5.9 Deleting ... 42
5.10 Exporting the Bundle ... 42
5.11 Data Plane Discovery ... 42
5.12 Engine Info ... 42
5.13 Two Deployment Workflows ... 43
6 Connecting to Azure Services ... 44
6.1 Prerequisites ... 44
6.2 Walkthrough: Service Bus Queue to Queue ... 44
6.3 Walkthrough: Event Hub ... 48
6.4 Direct HTTP (No Dapr) ... 49
6.5 Worked Example: Dynamics 365 to Peppol ... 49
6.6 Bindings vs. Pub/Sub: Which Pattern to Use ... 49
6.7 Summary: What You Configure Where ... 51
7 DTAP: From Development to Production ... 52
7.1 The Two Modes ... 52
7.2 Environment Layout ... 52
7.3 Development Workflow (Open Mode) ... 52
7.4 Promotion Pipeline ... 53
7.5 CI/CD: GitHub Actions Example ... 53
7.6 What Differs Per Environment ... 55
7.7 Same Queue Names, Different Namespaces ... 55
7.8 Operational Capabilities in Locked Mode ... 55
7.9 Rollback ... 56
7.10 Infrastructure as Code ... 56
8 Infrastructure as Code: Deploying UTLXe with Terraform and Bicep ... 57
8.1 What Gets Provisioned ... 57
8.2 Bicep: Complete Example ... 57
8.3 Terraform: Complete Example ... 62
8.4 CI/CD: Full Pipeline with Infrastructure + Bundle ... 66
8.5 Key Decisions ... 69
9 CI/CD Integration ... 70
9.1 The Deployment Model ... 70
9.2 Repository Structure ... 70
9.3 Building the Bundle ... 70
9.4 GitHub Actions Workflow ... 70
9.5 Azure DevOps Pipeline ... 72
9.6 Rollback ... 72
9.7 Multi-Environment Promotion ... 73
9.8 The Full Cycle ... 73
10 Persistence and Scaling ... 74
10.1 The Persistence Problem ... 74
10.2 Azure Files Volume Mount ... 74
10.3 CI/CD Re-Deploy Pattern ... 74
10.4 Startup Sequence ... 74
10.5 Memory Sizing ... 75
10.6 Horizontal Scaling ... 75
10.7 Never Use Swap ... 76
10.8 Poison Messages ... 76
11 Security ... 77

11.1 Network Architecture ... 77
11.2 Admin API Authentication ... 77
11.3 Non-Root Container ... 77
11.4 Secrets in Transformations ... 78
11.5 VNet Integration ... 78
11.6 TLS and HTTPS ... 78
12 Operations ... 79
12.1 Updating a Transformation ... 79
12.2 Pause and Resume ... 79
12.3 Incident Response Workflow ... 80
12.4 Validation Override ... 80
12.5 Updating Schemas ... 81
12.6 Bulk Operations ... 81
12.7 Engine Configuration ... 82
12.8 Production Locked Mode ... 82
12.9 Runtime Log Management ... 82
12.10 Messaging Configuration ... 83
13 Monitoring and Observability ... 84
13.1 Health Endpoint ... 84
13.2 Prometheus Metrics ... 84
13.3 Three Monitoring Options ... 85
13.4 Option 1: Azure Monitor Managed Prometheus + Managed Grafana ... 85
13.5 0 for 2 min ... 87
13.6 Option 2: Self-Hosted Prometheus + Grafana ... 87
13.7 Option 3: Admin API Only (No External Tools) ... 88
13.8 Monitoring Multiple UTLXe Instances ... 89
13.9 Error Diagnosis ... 89
13.10 Runtime Log Access ... 89
13.11 Metrics Design: What UTLXe provides vs. what's external ... 90
14 Troubleshooting ... 91
14.1 Container Starts But `ready=false` ... 91
14.2 Transformation Upload Fails (400) ... 91
14.3 Messages Failing (500 on Data Plane) ... 91
14.4 High Latency ... 91
14.5 Out of Memory (OOM Kill) ... 92
14.6 Admin API Returns 403 ... 92
14.7 Transformations Lost After Restart ... 92
14.8 Dapr Not Delivering Messages ... 92
14.9 Schema Validation Failing Unexpectedly ... 93
14.10 Diagnostic Commands Reference ... 93
15 Roadmap: What's Next ... 94
15.1 UTLXc: The Control Plane ... 94
15.2 .NET SDK and BizTalk Integration ... 95
15.3 Encoding Support (UTF-16, Shift-JIS) ... 95
15.4 OpenTelemetry ... 95
15.5 What You Can Do Today ... 95
16 Appendix A: API Reference ... 96
16.1 Authentication ... 96
16.2 Admin Port (8081) ... 96
16.3 Data Plane Port (8085) ... 98

16.4 Production Locked Mode (EF09) ... 98
16.5 Response Status Codes ... 98
17 Appendix B: Configuration Reference ... 100
17.1 Environment Variables ... 100
17.2 Engine Configuration (engine.yaml) ... 100
17.3 Port Map ... 100
17.4 Transformation Configuration (transform.yaml) ... 100
17.5 JVM Tuning Reference ... 101
17.6 Container Plans ... 101
17.7 Bundle ZIP Format ... 101
17.8 Data Directory Layout ... 101
18 Appendix C: UTL-X Quick Reference ... 103
18.1 File Structure ... 103
18.2 Format Headers ... 103
18.3 Property Access ... 103
18.4 Object Construction ... 103
18.5 Let Bindings ... 103
18.6 Conditionals ... 103
18.7 Array Operations ... 104
18.8 Pipe Operator ... 104
18.9 String Functions ... 104
18.10 Math Functions ... 104
18.11 Type Functions ... 104
18.12 Date Functions ... 105
18.13 Security Functions ... 105
18.14 User-Defined Functions ... 105

1 What UTLXe Does for Your Business

1.1 The Problem

Every enterprise connects systems that speak different data languages. An ERP sends invoices in JSON. A logistics partner expects XML. A warehouse system needs CSV. A government portal requires UBL 2.1. An IoT platform streams Event Hub telemetry that must be normalized before analysis.

Today, this translation is done by:

- **Custom code** — developers write integration scripts per connection. Each one is unique, untested, undocumented, and maintained by the person who wrote it.
- **Enterprise middleware** — MuleSoft, Tibco, SAP CPI, BizTalk. Powerful but expensive (six-figure licenses), complex to operate, and slow to change.
- **Manual processes** — someone downloads a file, opens Excel, reformats it, and uploads it elsewhere. Error-prone, unscalable, invisible.

The cost is not just the license or the developer hours. It is the **invisible cost**: messages lost between systems, invoices rejected because a field was in the wrong format, shipments delayed because the warehouse system could not parse the order.

1.2 What UTLXe Is

UTLXe is a **data transformation engine** that runs on Azure. You tell it how to translate one format to another using a simple, readable language (UTL-X). It does the rest — receiving messages, transforming them, and sending the results to the next system.

What you do	What UTLXe does
Write a transformation rule (10-50 lines)	Compile it into an optimized runtime
Connect a Service Bus queue	Receive messages automatically via Dapr
Deploy via Azure Marketplace	Run as a managed container — no servers to maintain
Monitor via dashboard	Track messages, errors, latency in real time

A transformation looks like this:

```
%utlx 1.0
input json
output xml
---
<Invoice>
  <ID>{$input.invoiceId}</ID>
  <BuyerName>{upperCase($input.customer.name)}</BuyerName>
  <Total>{round($input.amount * 1.21, 2)}</Total>
</Invoice>
```

That is the entire program. No boilerplate. No framework. No build step. Upload it, and messages start transforming.

1.3 Who Uses UTLXe

Role	What they care about	What UTLXe gives them
IT Manager	Cost, reliability, time-to-deploy	Azure Marketplace deploy in minutes. No license negotiation. Pay per container.

Integration Developer	Productivity, correctness, debugging	Simple language, instant test, error ring buffer, hot-swap without restart.
Operations	Uptime, monitoring, incident response	Prometheus metrics, Grafana dashboards, pause/resume per transformation.
Architect	Standards, security, scalability	Managed Identity, VNet isolation, immutable production bundles, CI/CD integration.
Business Analyst	Understanding what transforms happen	Readable `.utlx` rules — not code hidden in a Java class.

1.4 Compared to Alternatives

Aspect	**Custom code**	**MuleSoft / Tibco**	**Azure Logic Apps**	**UTLXe**
Deploy time	Weeks	Days–weeks	Hours	**Minutes**
Format support	Whatever you code	Broad	JSON + XML	**JSON, XML, CSV, YAML, Avro, Protobuf**
Cost	Developer hours	License + infra	Per execution	**Per container (fixed)**
Language	Java / C# / Python	DataWeave / XSLT	Visual designer	**UTL-X (readable, auditable)**
Testing	Write your own	Studio	Limited	**Built-in test endpoint**
Hot-swap	Redeploy	Redeploy	Save	**Instant, zero-downtime**
Monitoring	Build it	Included	Basic	**Prometheus + Grafana + Admin API**
Lock-down	Custom	Platform	None	**Immutable .utlar bundles**

1.5 Azure Marketplace: What You Get

When you deploy UTLXe from the Azure Marketplace, the following is provisioned automatically:

1. **UTLXe container** — the transformation engine, ready to receive rules.
2. **Dapr sidecar** — connects to Azure Service Bus, Event Hub, or any Dapr-supported broker. No custom networking.
3. **Admin Web UI** (optional) — browser-based interface for uploading transformations, testing, and monitoring.
4. **Persistent storage** (optional) — Azure Files mount so transformations survive container restarts.
5. **Health and metrics** — Prometheus endpoint for Grafana or Azure Monitor.

The wizard asks three questions: resource group, Service Bus connection, and an admin password. Five minutes later, you have a running transformation engine.

1.6 Business Scenarios

1.6.1 E-Invoicing (Peppol / UBL)

European e-invoicing regulations require invoices in UBL 2.1 XML format. Your ERP (Dynamics 365, SAP, or a custom system) produces JSON. UTLXe transforms JSON invoices to UBL XML in real time, ready for Peppol delivery.

1.6.2 EDI Modernization

Legacy EDI (EDIFACT, X12) can be converted to modern JSON/XML formats for downstream systems. Instead of maintaining decades-old EDI translators, write a 30-line UTL-X rule.

1.6.3 IoT Data Normalization

Sensors publish telemetry in varying formats via Event Hub. UTLXe normalizes the data into a consistent schema before it reaches your analytics platform.

1.6.4 Multi-System Order Processing

An e-commerce order arrives as JSON. The warehouse needs CSV. The accounting system needs XML. The shipping partner needs a specific JSON structure. UTLXe runs four transformations in parallel — one input, four outputs.

1.6.5 API Gateway Transformation

An external API returns data in a format your frontend cannot consume. UTLXe sits behind Azure API Management and transforms the response on the fly.

1.7 Pricing Model

UTLXe runs as an Azure Container App. You pay for the container resources (CPU + memory), not per message or per transformation. This makes costs predictable:

Plan	Container	Suitable for
Starter	1 vCPU, 2 GB RAM	Up to 50 messages/sec, small payloads
Professional	2 vCPU, 4 GB RAM	Up to 200 messages/sec, larger payloads

Scale horizontally by adding more container instances. Each instance handles its own set of queues — no shared state, no coordination overhead.

No license fees. No per-message charges. No minimum commitment. Stop the container, stop paying.

1.8 What This Book Covers

The rest of this book is a hands-on guide for deploying and operating UTLXe on Azure:

- **Chapters 2–5**: Getting started — quick deploy, write transformations, connect to Azure services.
- **Chapters 6–8**: Automation — environment strategy (Dev/Test/Acc/Prd), infrastructure as code, CI/CD pipelines.
- **Chapters 9–13**: Production — persistence, security, operations, monitoring, troubleshooting.
- **Chapter 14**: Roadmap — control plane, .NET SDK, and what is coming next.
- **Appendices**: Complete API reference, configuration reference, UTL-X quick reference.

Start with Chapter 2 (Quick Start) to see UTLXe running in five minutes.

2 Why UTLXe?

Every integration project has the same problem: data arrives in one format and needs to leave in another. An order from Dynamics 365 is JSON. The Peppol e-invoicing network expects UBL XML. A warehouse system sends CSV. An IoT platform produces YAML. The business logic is often simple — rename a field, calculate a total, filter a list — but the plumbing to connect these systems is not.

This chapter explains where UTLXe fits in an Azure middleware architecture, what problems it solves, and why those problems are harder than they appear.

2.1 The Transformation Problem

Consider a common Azure integration: Dynamics 365 Business Central publishes sales orders to Azure Service Bus. A downstream system consumes them and expects a different structure.

Without UTLXe, you write the transformation in one of these places:

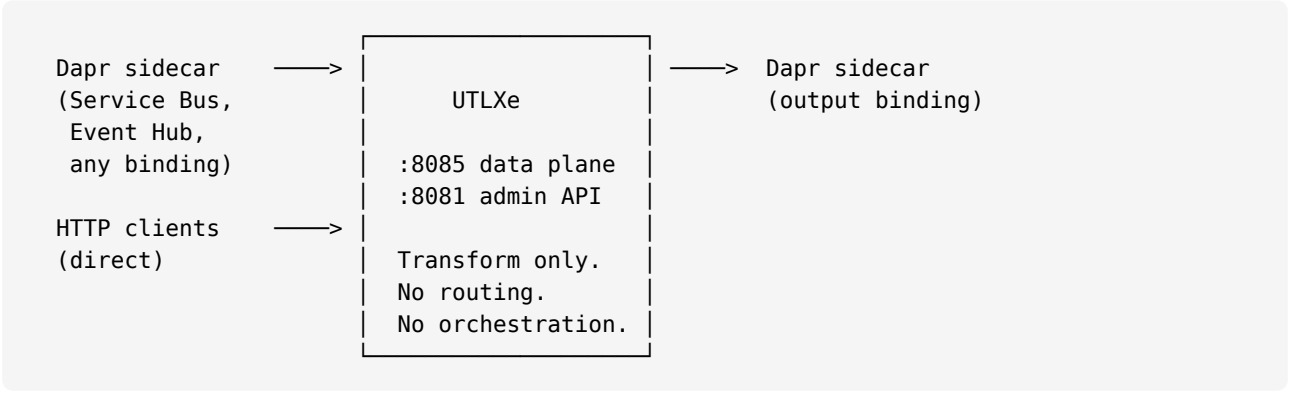
Approach	How	Problem
Azure Function	C# or Python code that parses JSON, builds output, serializes	Business logic buried in infrastructure code. Every mapping change requires a code deployment.
Logic App	Visual designer with built-in transforms	Limited expression language. Complex mappings become unreadable. No version control for visual flows.
API Management policy	Liquid templates or XSLT in the gateway	Mixing transformation with routing. Templates are hard to test. No debugging.
In the producer	D365 emits the target format directly	Tight coupling. Producer must know every consumer's format. Changes ripple upstream.
In the consumer	Downstream system accepts the raw format	Same coupling problem in reverse. Every consumer must handle every producer's format.

All of these approaches scatter transformation logic across the architecture. When the Peppol specification changes, or a new trading partner requires a different XML dialect, or the warehouse system switches from CSV to JSON, someone has to find the right Azure Function, Logic App, or XSLT template and modify it.

2.2 What the UTLXe Azure Offering Does

UTLXe is a dedicated transformation engine that runs as a container on Azure Container Apps. Its only job is to transform data — parse an incoming message, apply mapping logic, and produce output in the required format.

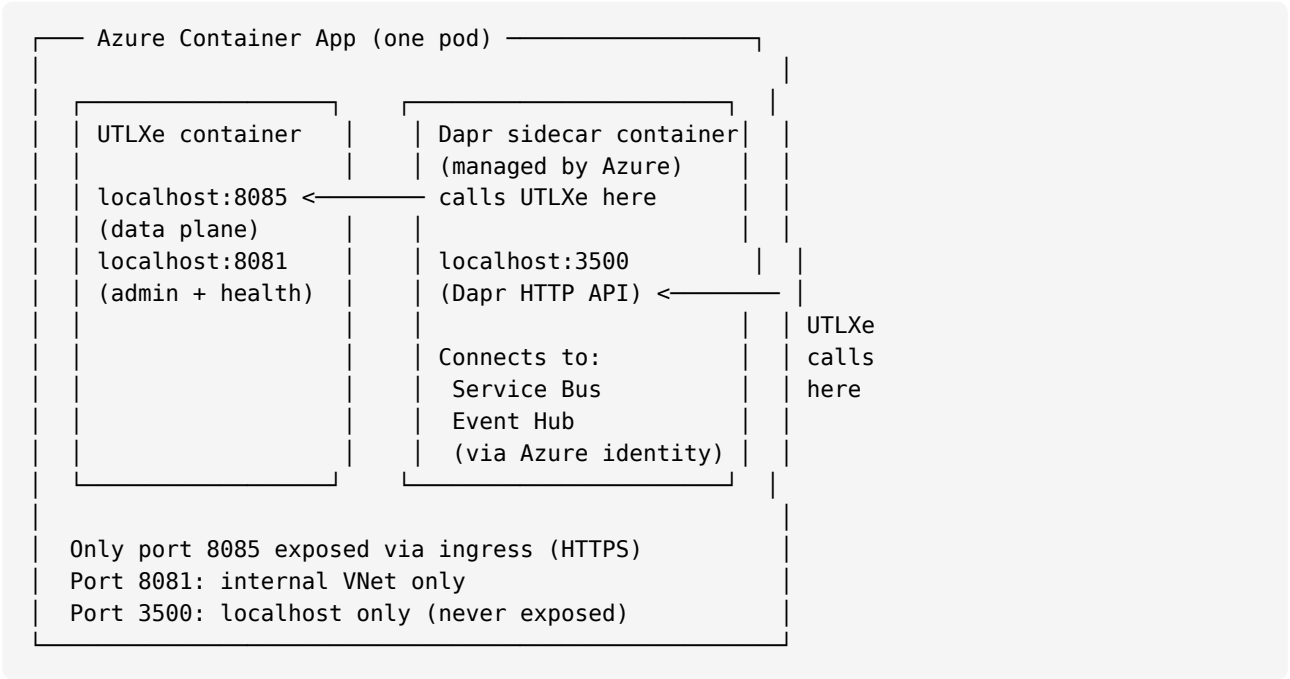
UTLXe exposes two HTTP ports. That is its entire external interface:



Port 8085 is the data plane — messages in, results out. Port 8081 is the admin API — upload transformations, check health, view metrics. UTLXe never connects to Azure services directly. It receives and sends HTTP. The connection to messaging infrastructure is handled by a Dapr sidecar.

The following sections show each connection scenario with a concrete example.

Before showing the scenarios, it helps to understand what runs where. Azure Container Apps deploys UTLXe and Dapr as **two separate containers in the same pod**. They share a `localhost` network — they can call each other on `localhost` without any network traversal or firewall rules.



Key points:

- **Port 8085** (UTLXe data plane): exposed via Container App ingress with HTTPS. Dapr calls it on `localhost:8085` from inside the pod. External clients reach it via the ingress URL.
- **Port 8081** (UTLXe admin): internal to the VNet. Not exposed via ingress. Used by operators and CI/CD.

- **Port 3500** (Dapr HTTP API): localhost only, never leaves the pod. UTLXe calls it to send output messages. No firewall rule needed — it is pod-internal.
- Dapr connects to Service Bus and Event Hub using Azure Managed Identity or connection strings stored as Container App secrets. UTLXe never sees these credentials.

2.2.1 Startup: How Dapr and UTLXe Initialize

Before any messages flow, the Dapr sidecar and UTLXe go through a startup handshake. This happens once when the container starts. Note that transformations may not be uploaded yet at this point — the Admin API accepts uploads at any time.

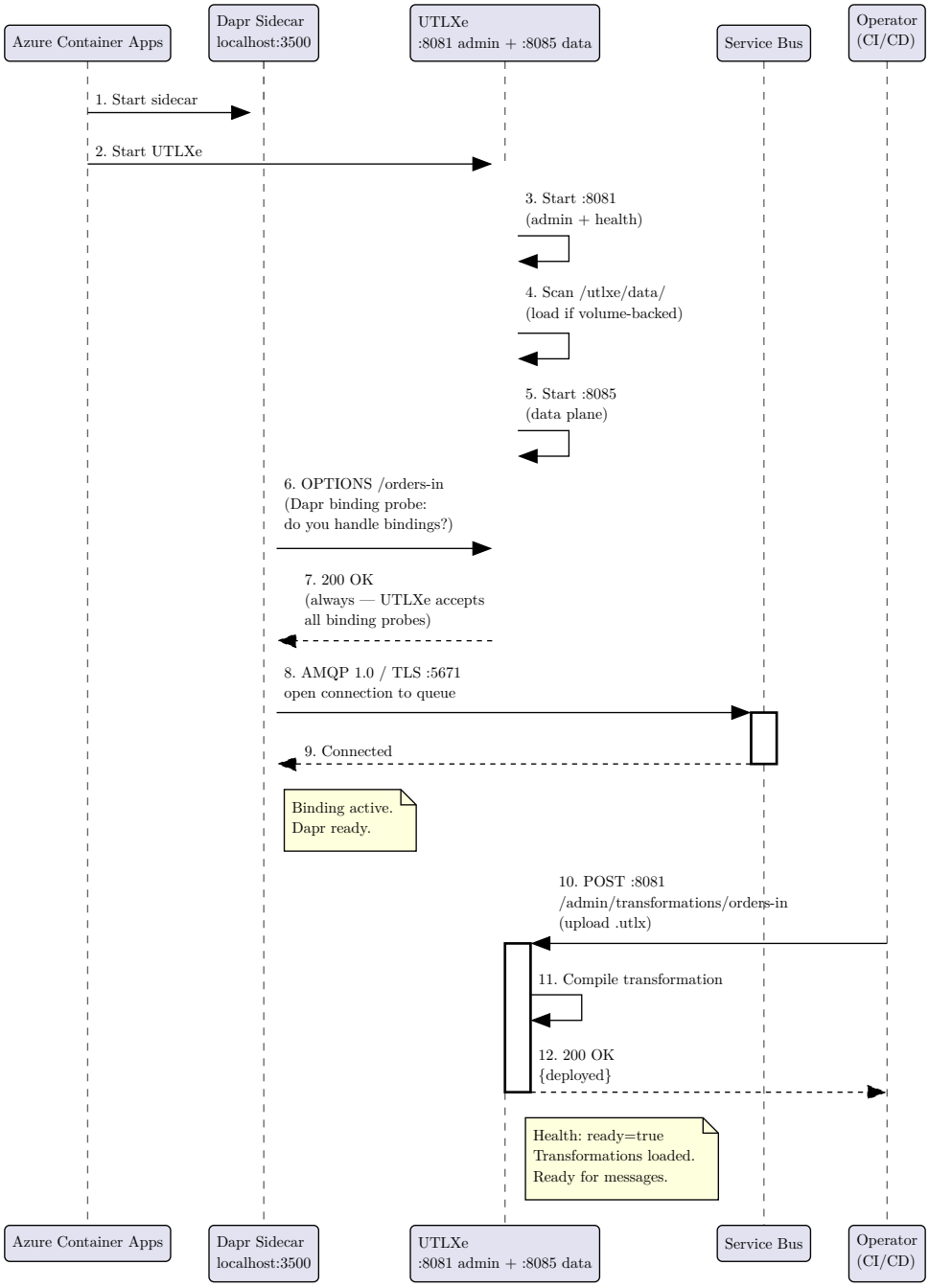


Figure 1: Startup sequence: Dapr probe, Admin API upload, then ready for messages

This diagram shows **open mode** (dev/test) — where transformations are uploaded via the Admin API after startup.

Step by step:

1. Azure Container Apps starts both containers in the same pod (UTLXe and Dapr sidecar).
2. UTLXe starts the admin API on port 8081 — health checks, metrics, and bundle management are immediately available.
3. UTLXe scans `/utlxe/data/` — if Azure Files is mounted with transformations from a previous session, they are compiled now. If a `bundle.utlar` file is found, UTLXe enters **locked mode** instead (see below).
4. UTLXe starts the data plane on port 8085.
5. Dapr sends an `OPTIONS /orders-in` probe to port 8085. This is Dapr asking: "Do you handle bindings?" The `OPTIONS` method is an HTTP verb (like GET or POST) used to check capabilities.
6. UTLXe responds 200 OK — **always**, regardless of whether the `orders-in` transformation exists yet. This tells Dapr "yes, I handle bindings." The actual transformation check happens when the first message arrives.
7. Dapr opens an AMQP connection to the Service Bus queue.
8. The operator (or CI/CD pipeline) uploads the transformation via the Admin API: `POST :8081/admin/transformations/orders-in`.
9. UTLXe compiles the transformation and reports `ready: true`.

Why does UTLXe always respond 200 to the OPTIONS probe? Because the Admin API upload (step 10) may happen after Dapr's probe (step 6). If UTLXe returned 404 for a not-yet-uploaded transformation, the binding would never activate — and the operator couldn't fix it without restarting the container. By responding 200 always, Dapr activates the binding immediately. Messages that arrive before the transformation is uploaded get a 503 response, which triggers Service Bus retry. Once the transformation is uploaded, the next retry succeeds.

2.2.2 Startup: Locked Mode (Production with `.utlar`)

In production, transformations are deployed via CI/CD as an immutable `.utlar` archive. When UTLXe finds `bundle.utlar` on the data volume, the startup sequence is different:

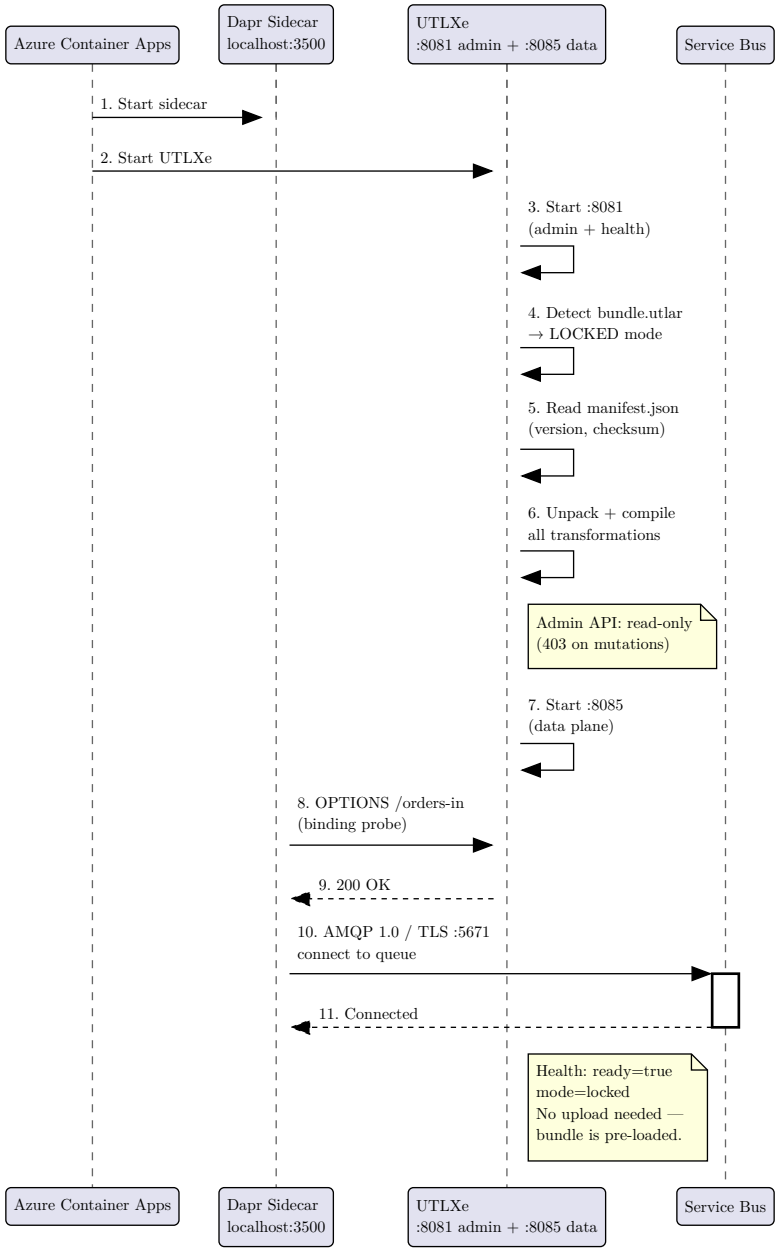


Figure 2: Locked mode startup: `.utlar` pre-loaded by CI/CD, no Admin API upload needed

The key difference: in locked mode, transformations are already compiled before Dapr probes. The Admin API never receives an upload — changes go through CI/CD. Operational endpoints (pause, resume, validation override, log management) remain available for incident response.

2.2.2.1 Multiple queues, multiple bindings

When you have multiple queues, each queue is a separate Dapr component. Dapr probes each one at startup:

```
OPTIONS /orders-in      → 200 OK (binding activated)
OPTIONS /invoices-in    → 200 OK (binding activated)
OPTIONS /returns-in     → 200 OK (binding activated)
```

Each binding maps to a transformation with the same name. The customer uploads all transformations via the Admin API (or as a bundle):

Service Bus queue	Dapr component	UTLXe transformation
`incoming-orders`	`orders-in`	`orders-in.utlx`
`incoming-invoices`	`invoices-in`	`invoices-in.utlx`
`incoming-returns`	`returns-in`	`returns-in.utlx`

To verify the binding matches, use the validation endpoint:

```
curl -H "X-Admin-Key: $KEY" http://<admin>:8081/admin/dapr/bindings
```

```
{
  "bindings": [
    {"binding": "orders-in", "transformation": "orders-in", "status": "matched"},
    {"binding": "invoices-in", "transformation": "invoices-in", "status": "matched"},
    {"binding": "returns-in", "transformation": null, "status": "MISSING"}
  ]
}
```

A `MISSING` status means Dapr activated the binding but no transformation was uploaded for it. Messages on that queue will fail (500) until the transformation is uploaded.

The following scenarios show the steady-state message flow — after startup is complete.

2.2.3 Scenario 1: Azure Service Bus via Dapr

The most common pattern. Messages arrive on a Service Bus queue, get transformed, and are forwarded to an output queue.

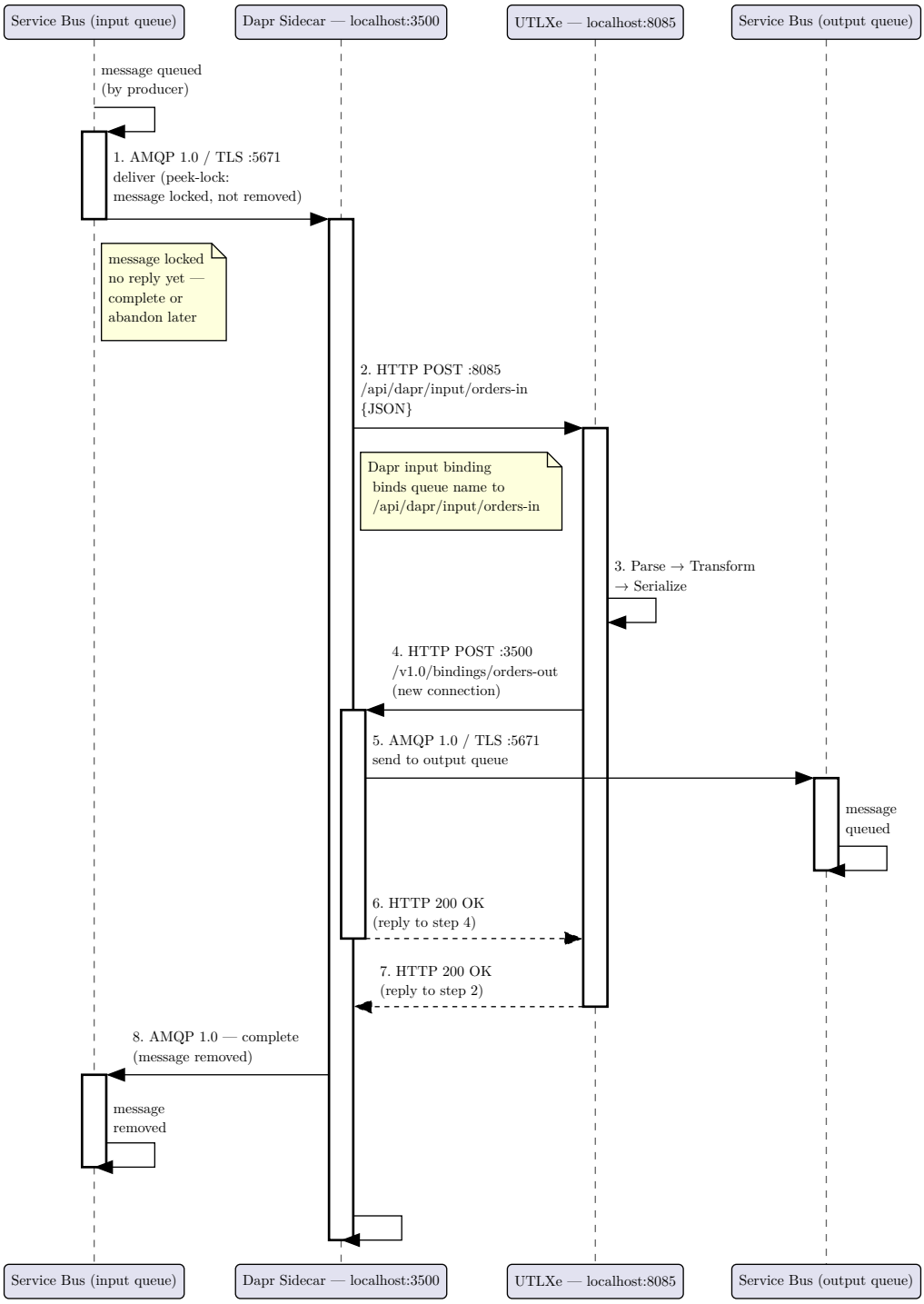


Figure 3: Scenario 1: Azure Service Bus queue-to-queue via Dapr sidecar

Step by step:

1. Service Bus delivers the message to Dapr over AMQP using **peek-lock** mode. The message is **locked** (invisible to other consumers) but **not removed**. There is no reply at this point — the lock is held until Dapr

sends a "complete" or "abandon" disposition later (step 8). The lock has a configurable timeout (default 30 seconds).
2. Dapr calls UTLXe: `POST localhost:8085/api/dapr/input/orders-in`. This HTTP request stays open until step 7.
3. UTLXe parses, transforms, and serializes the output.
4. UTLXe opens a **new, separate HTTP connection** to Dapr: `POST localhost:3500/v1.0/bindings/orders-out`. This is not a reply to step 2 — it is an independent outbound call.
5. Dapr forwards the output to the output Service Bus queue via AMQP.
6. Dapr returns HTTP 200 to UTLXe's request from step 4.
7. UTLXe returns HTTP 200 to Dapr's original request from step 2 — processing succeeded.
8. Dapr sends a **complete** disposition to Service Bus for the original message from step 1. The message is permanently removed from the input queue. If step 7 had returned HTTP 500, Dapr would send **abandon** instead — the lock is released, and Service Bus makes the message visible again for retry. After the maximum delivery count is exceeded, the message moves to the dead-letter queue.

No Azure SDK, no connection code, no retry logic in UTLXe. Dapr handles the AMQP connection to Service Bus (port 5671, TLS), peek-lock management, authentication, retries, and dead-letter routing. The customer configures only **what** to connect to — never **how**. Chapter 4 walks through the complete setup.

2.2.3.1 Rainy day: transformation fails

When UTLXe cannot transform a message (null reference, type mismatch, schema validation failure), it returns HTTP 500 instead of 200. This triggers the retry and dead-letter path:

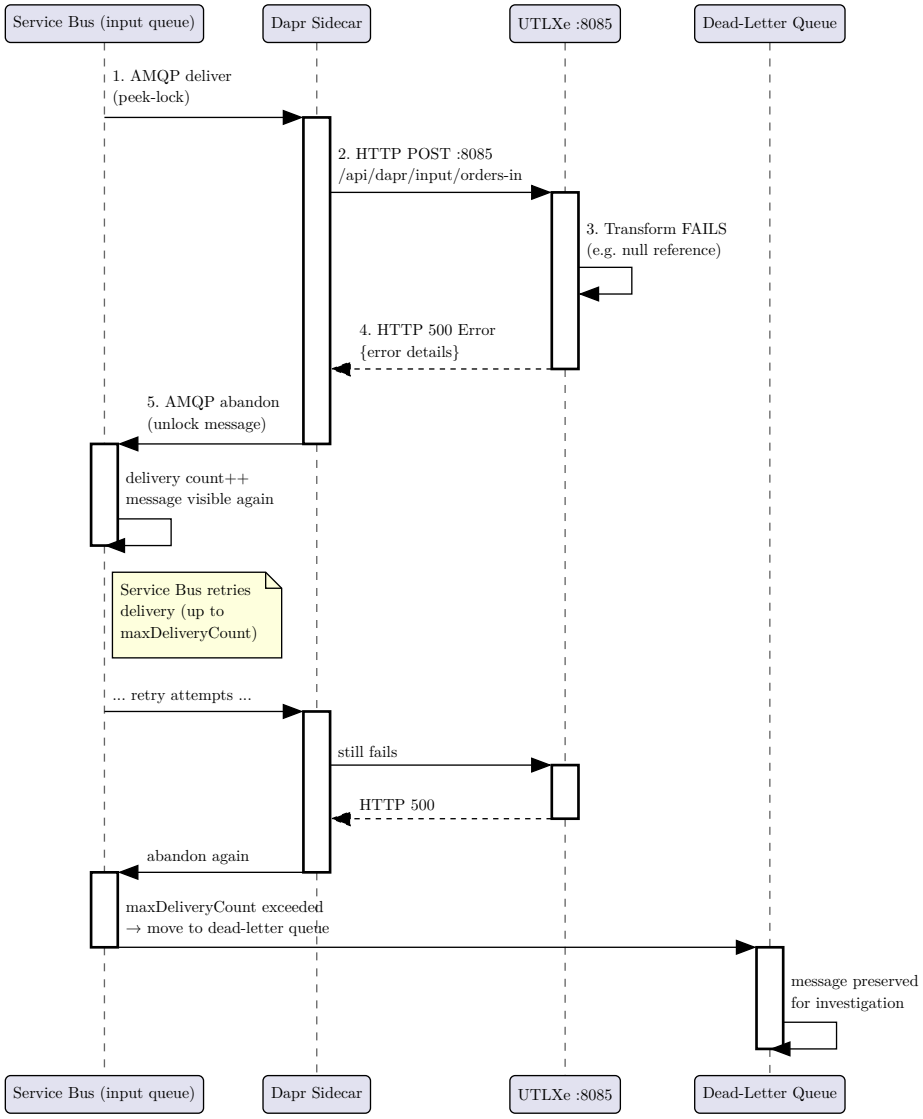


Figure 4: Rainy day: transformation fails → retry → dead-letter queue

When a transformation fails:

- UTLXe returns HTTP 500 with error details (line number, error message).
- Dapr **abandons** the message on Service Bus — the peek-lock is released, and the message becomes visible again.
- Service Bus increments the delivery count and redelivers the message.
- After `maxDeliveryCount` attempts (configurable on the queue, default 10), Service Bus moves the message to the **dead-letter queue** (DLQ).
- The DLQ preserves the original message for investigation. Use the Admin API (`GET /admin/transformations/{name}/errors`) to see recent error details.

The Event Hub rainy day is simpler: there is no dead-letter queue. If UTLXe returns 500, Dapr does not checkpoint the offset. The event is redelivered on the next consumer restart. Persistent failures require manual intervention (pause the transformation, fix, resume).

Dapr connects to Service Bus using YAML component definitions. The **component name** determines which UTLXe transformation is called — Dapr posts to `/{component-name}` on the app port, and UTLXe looks up a transformation with that name.

At startup, Dapr sends an `OPTIONS /{component-name}` probe to verify UTLXe is listening. UTLXe responds with 200, and Dapr activates the binding. No `route` metadata is needed — the component name IS the route.

```
# Dapr input binding — component name "orders-in"
componentType: bindings.azure.servicebusqueues
metadata:
  - name: connectionString
    secretRef: servicebus-connection
  - name: queueName
    value: "incoming-orders"
  - name: route
    value: "/orders-in"       # binds this queue to transformation "orders-in"
scopes: [utlxe]
```

The `route` metadata makes the binding explicit: queue `incoming-orders` → route `/orders-in` → transformation `orders-in`. Without it, Dapr uses the component name as the path (same result, but implicit). The `route` is recommended because it makes the binding visible in one place — you can read the YAML and know exactly which transformation handles this queue.

```
# Dapr output binding — component name "orders-out"
# UTLXe calls: POST localhost:3500/v1.0/bindings/orders-out
componentType: bindings.azure.servicebusqueues
metadata:
  - name: connectionString
    secretRef: servicebus-connection
  - name: queueName
    value: "processed-orders"
scopes: [utlxe]
```

The output binding name (`orders-out`) is configured in the transformation's `transform.yaml` as `outputBinding: orders-out`. UTLXe calls Dapr's standard binding API at `localhost:3500/v1.0/bindings/orders-out`.

The complete wiring:

Configuration	Where	Connects
`route: /orders-in`	Dapr input component	Service Bus queue → `POST /orders-in` → UTLXe transformation `orders-in`
`outputBinding: orders-out`	UTLXe transform.yaml	UTLXe → `POST :3500/v1.0/bindings/orders-out` → Dapr
`queueName: processed-orders`	Dapr output component	Dapr output binding → Service Bus output queue
`appPort: 8085`	Container App Dapr config	Dapr sidecar → UTLXe HTTP port

2.2.3.2 How UTLXe knows the output binding

The output binding name (`orders-out`) is not passed by Dapr in the input call. It is pre-configured on the UTLXe side, in the transformation's `transform.yaml`:

```
outputBinding: orders-out
```

UTLXe resolves the output binding from three sources (highest priority first): an environment variable override (`UTLXE_OUTPUT_BINDING_ORDERS_IN`), the `transform.yaml` config, or an `X-UTLXe-Output-Binding` HTTP header. The most common is the config file.

2.2.3.3 Concurrency and correlation

When multiple messages are processed concurrently (multiple Dapr calls to UTLXe at the same time), there is no correlation problem. Each message is handled on its own pair of HTTP connections:

```
Worker 1:  conn A: Dapr──POST :8085──>UTLXe (open)
           conn B: UTLXe──POST :3500──>Dapr (new, independent)
           conn A: UTLXe──200 OK──>Dapr (same TCP connection)

Worker 2:  conn C: Dapr──POST :8085──>UTLXe (open)
           conn D: UTLXe──POST :3500──>Dapr (new, independent)
           conn C: UTLXe──200 OK──>Dapr (same TCP connection)
```

HTTP is connection-based — the 200 response goes back on the **same TCP connection** that Dapr opened. There is no cross-worker confusion. The output binding call (connection B/D) is completely independent — Dapr does not need to match it to the input. The only thing that determines the input message's fate (complete vs. abandon) is the HTTP status code returned on the original connection.

If message ordering matters, set `maxConcurrentHandlers: 1` in the Dapr input component metadata, or use Service Bus sessions. But that is a Service Bus concern, not a UTLXe or Dapr correlation concern.

2.2.3.4 Message tracing and correlation

In production, you need to trace which input message produced which output message. UTLXe preserves the following metadata across the transformation:

UTLXe implements the standard messaging triad:

ID	Constant?	What happens
`MessageId`	No	New UUID generated for each output message.
`CorrelationId`	Yes	Preserved from input. Links all messages in the same business process.
`CausationId`	No	Set to the input's `MessageId`. Links each output to its immediate cause.
`traceparent`	—	W3C Trace Context forwarded. Azure Monitor shows the full distributed trace.

In a multi-step chain, the `CorrelationId` stays constant while the `CausationId` traces the parent:

All IDs are UUIDv7 (RFC 9562) — time-ordered with an embedded timestamp, so IDs are sortable by creation time:

```
Producer:   MessageId=UUID-A  CorrelationId=UUID-CORR  CausationId=(none)
UTLXe:      MessageId=UUID-B  CorrelationId=UUID-CORR  CausationId=UUID-A
Next step:  MessageId=UUID-C  CorrelationId=UUID-CORR  CausationId=UUID-B
```

Every processed message is logged with all three IDs:

```
INFO [orders-in] MessageId=UUID-A  CorrelationId=UUID-CORR
     → transformed in 3ms → output MessageId=UUID-B
     CausationId=UUID-A
```

Error entries in the Admin API (`GET /admin/transformations/{name}/errors`) also include all three IDs, so you can correlate a failed transformation back to the specific input message on Service Bus.

2.2.4 Scenario 2: Azure Event Hub via Dapr

Same 8-step flow, different Dapr component type. UTLXe does not know the difference.

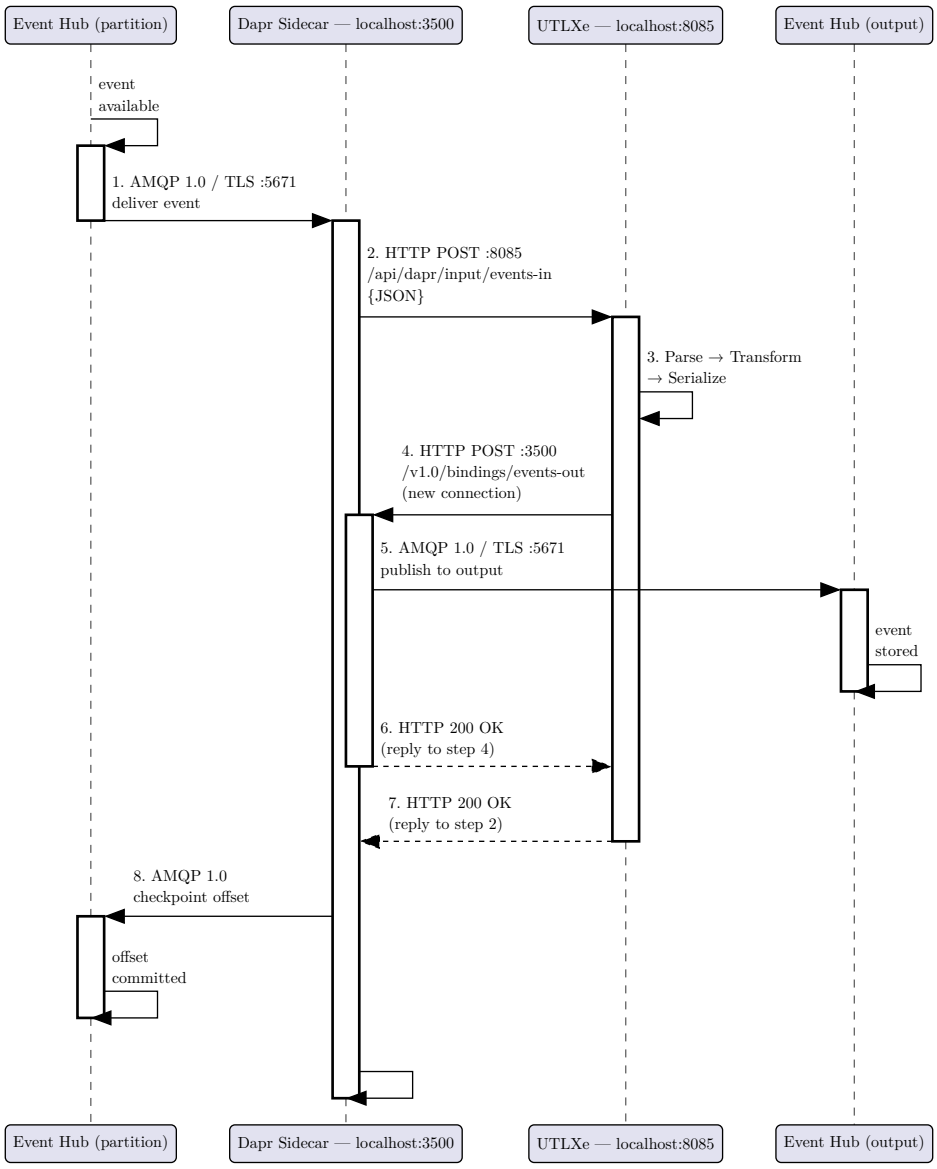


Figure 5: Scenario 2: Azure Event Hub via Dapr sidecar

The only difference from Scenario 1 is the Dapr component type and the checkpointing mechanism. Event Hub uses a storage account for offset tracking. The UTLXe transformation is identical.

```
# Dapr input binding — Event Hub
componentType: bindings.azure.eventhubs
metadata:
  - name: connectionString
    secretRef: eventhub-connection
  - name: consumerGroup
    value: "utlxe-consumer"
  - name: storageAccountName
    value: "stutlxecheckpoint"
  - name: storageContainerName
```

```
    value: "checkpoints"
scopes: [utlxe]
```

2.2.5 Scenario 3: Direct HTTP (no Dapr)

For synchronous request/response patterns — API gateways, batch scripts, testing — clients call UTLXe directly via the Azure ingress. No Dapr sidecar, no message queue, no YAML configuration.

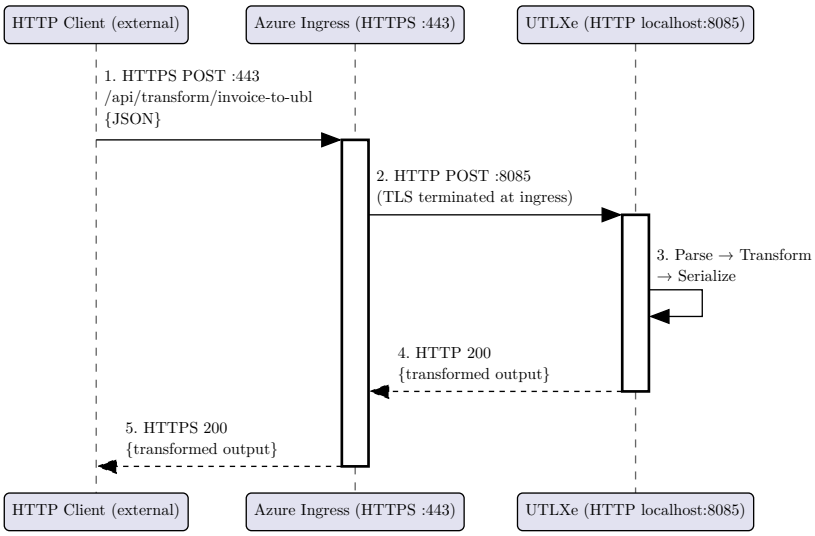


Figure 6: Scenario 3: Direct HTTP via Azure ingress, no Dapr

```
curl -X POST \
  -H "Content-Type: application/json" \
  -H "X-Message-Id: <UUID-A>" \
  -H "X-Correlation-Id: <UUID-CORR>" \
  -H "X-Causation-Id: <UUID-PREV>" \
  -d '{"orderNumber":"12345","amount":200.00}' \
  https://myapp.azurecontainerapps.io/api/transform/invoice-to-ubl
```

Direct HTTP clients can send `X-Message-Id`, `X-Correlation-Id`, and `X-Causation-Id` headers for traceability. If `X-Message-Id` is omitted, UTLXe generates a UUID — every request is traceable, even ad-hoc curl calls. The response echoes the correlation headers:

```
HTTP/1.1 200 OK
X-Message-Id: <UUID-B>
X-Correlation-Id: <UUID-CORR>
X-Causation-Id: <UUID-A>
X-Transform-Duration-Ms: 3
```

No Dapr, no Service Bus. A direct HTTPS call, a transformed response. Azure ingress handles TLS — UTLXe receives plain HTTP on localhost:8085.

2.2.5.1 Rainy day: direct HTTP error

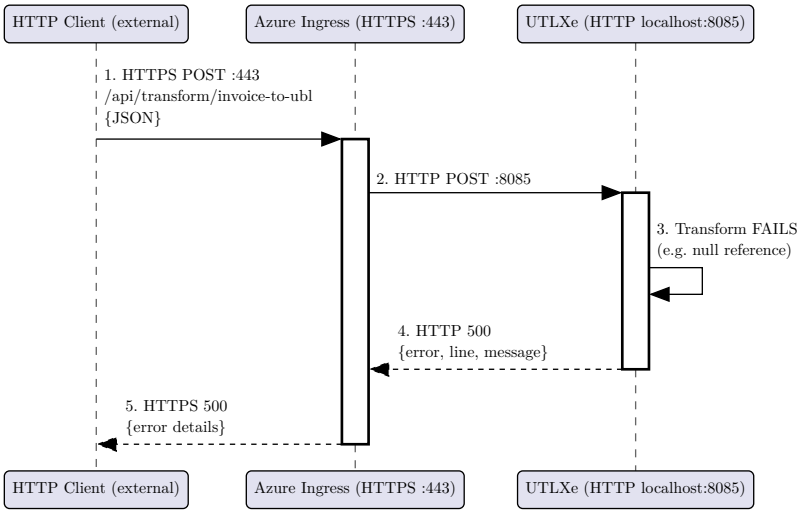


Figure 7: Rainy day: direct HTTP — error returned to client

With direct HTTP, the error goes straight back to the caller. There is no retry, no dead-letter queue — the client decides what to do (retry, log, alert). The error response includes the transformation name, line number, and error message.

2.2.6 The Transformation

In all three scenarios, the transformation is the same `.utlx` file:

```
%utlx 1.0
input json
output xml
---
{
  Invoice: {
    ID: concat("INV-", $input.orderNumber),
    IssueDate: formatDate(now(), "yyyy-MM-dd"),
    BuyerParty: { Name: $input.customer.name },
    Total: round($input.amount * 1.21, 2)
  }
}
```

The transformation does not contain any reference to Service Bus, Event Hub, or HTTP. It expresses only the mapping. The transport is a deployment concern, not a transformation concern.

This pattern applies wherever data crosses a format or schema boundary: e-invoicing (Peppol, UBL), API format conversion, IoT sensor normalization, ERP migration, and B2B partner integration.

UTLXe is capable of much more than what this Azure offering exposes — including gRPC transport, protobuf-based language wrappers for .NET, Go, and Python, Kafka integration, and pipeline chaining. For the full capabilities, see the UTLXe documentation at `https://github.com/grauwen/utl-x`. This book covers only the Azure Marketplace deployment.

2.3 The Cost of Embedded Transformation

To understand why a dedicated engine matters, consider what happens when transformation logic is embedded in an Azure Function:

```
// OrderToInvoice.cs — 180 lines
public static async Task<IActionResult> Run(
    [ServiceBusTrigger("orders")] string message,
    [ServiceBus("invoices")] IAsyncCollector<string> output)
{
    var order = JsonConvert.DeserializeObject<Order>(message);
    var invoice = new Invoice {
        ID = "INV-" + order.OrderNumber,
        IssueDate = DateTime.UtcNow.ToString("yyyy-MM-dd"),
        // ... 150 more lines of mapping logic
    };
    var xml = new XmlSerializer(typeof(Invoice))...
    await output.AddAsync(xml);
}
```

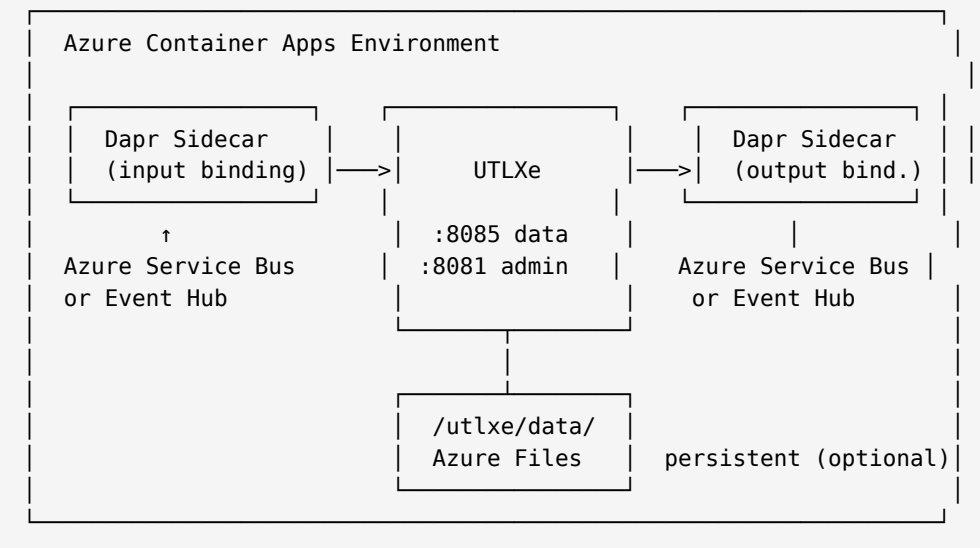
This works, but:

- The mapping logic is mixed with the infrastructure (Service Bus trigger, serialization, error handling).
- Testing requires a running Service Bus emulator or mock.
- Changing one field mapping requires a full C# build and deployment.
- The developer needs to know C#, the Azure Functions SDK, and the XML serialization API — in addition to the business mapping rules.

With UTLXe, the same mapping is twelve lines of `.utlx`. No build step, no SDK, no deployment pipeline for the container. Upload the file and it works. Change a field, re-upload, zero downtime.

2.4 How UTLXe Runs on Azure

UTLXe is available as a pre-built container on the Azure Marketplace. You deploy it like any other Container App — no custom Docker image needed.



- **Azure Container Apps** runs the container and handles scaling, health checks, networking, and TLS.
- **Dapr sidecar** connects to Azure messaging services. Configured with YAML, no code.
- **UTLXe** transforms messages. Configured with `.utlx` files, no code.
- **Azure Files** (optional) persists uploaded transformations across container restarts.

2.5 The Deployment Workflow

Unlike Azure Functions or Logic Apps, UTLXe separates the container from the transformations. The container is deployed once from the Marketplace. The transformations are deployed independently via the Admin API:

1. **Deploy the container** from the Azure Marketplace (once).
2. **Upload transformations** via `POST /admin/bundle` or individual `POST /admin/transformations/{name}`.
3. **Test** each transformation with sample input via `POST /admin/transformations/{name}/test`.
4. **Send traffic** to the data plane on port 8085.
5. **Update transformations** at any time — upload a new version, zero downtime.

No Docker builds, no container restarts, no CI/CD pipeline for infrastructure. The pipeline only ships `.utlx` files.

2.6 What UTLXe Does Not Do

UTLXe is a transformation engine, not an integration platform. It deliberately does not route messages, orchestrate workflows, store data, or manage connections. Dapr handles messaging. Logic Apps handle orchestration. Azure Container Apps handles infrastructure. UTLXe handles transformation — and only transformation.

This narrowness is intentional. A tool that does one thing well composes better than a tool that does everything adequately.

2.7 When to Use UTLXe

Use UTLXe when:

- Data arrives in one format and must leave in another.
- Transformation logic changes more often than infrastructure.
- Multiple team members need to read and modify mappings.
- You want version-controlled, testable, reviewable transformation logic.
- You need zero-downtime updates for transformation rules.

Use Azure Functions or Logic Apps instead when:

- The integration involves orchestration (multi-step, conditional branching, human approval).
- The transformation is trivial (rename one field) and doesn't justify a dedicated engine.
- You need to call external APIs as part of the transformation (UTLXe is stateless and does not make outbound API calls during transformation).

2.8 Migrating from BizTalk Server

Microsoft BizTalk Server is approaching end of life. The SBMP protocol for Azure Service Bus retires September 30, 2026 — breaking BizTalk deployments that use the default Service Bus adapter. BizTalk 2020 extended support ends April 2030. Organizations running BizTalk are actively looking for a migration path.

UTLXe replaces the **transformation** piece of BizTalk — the pipeline component and XSLT/.NET map. It does not replace BizTalk's adapters, MessageBox, or orchestrations. Those migrate to Logic Apps Standard. The transformation logic — the part that took years to get right — migrates to UTLXe.

2.8.1 The three-step migration

1. **Today:** Drop UTLXe into existing BizTalk pipelines. Replace XSLT maps and custom .NET maps with `.utlx` transformations. No change to adapters, MessageBox, or orchestrations. The customer keeps running BizTalk, but their transformation logic is now portable.
2. **Migration window (2026–2030):** Migrate adapters and orchestrations to Logic Apps Standard at whatever pace fits the business. The transformation logic moves unchanged — the same `.utlx` files that ran inside BizTalk now run inside Logic Apps Standard custom functions or behind a UTLXe sidecar on Azure Container Apps.

3. **Steady state:** Logic Apps Standard for orchestration, UTLXe for transformation, optionally Dapr for cross-cloud portability. The customer's transformation IP is decoupled from their host platform forever.

2.8.2 Why this matters

The transformation logic is the most valuable and most fragile part of an integration. It encodes business rules, field mappings, tax calculations, format-specific quirks, and partner-specific exceptions. Rewriting it in a new language (DataWeave, Liquid, C#) is expensive and risky.

UTLXe lets the customer **extract** the transformation logic into a portable, testable, version-controlled format — and then move the host underneath it. BizTalk today, Logic Apps tomorrow, Kubernetes next year. The `.utlx` files stay the same.

2.8.3 Integration paths

BizTalk pipeline	A .NET SDK shim (`UtlxEngine.BizTalk`) implements `IComponent`. Drop into a pipeline stage, set the bundle and rule name, done. Same SDK that runs the engine subprocess over stdin/stdout protobuf.
Logic Apps Standard	A custom .NET 8 function (`UtlxEngine.LogicApps`) calls UTLXe via the SDK. Register with dependency injection, invoke from any workflow action.
Azure Container Apps	UTLXe runs as a container with Dapr sidecar. Messages arrive from Service Bus or Event Hub, get transformed, and are forwarded. No .NET code needed — just `.utlx` files and YAML configuration.
Direct HTTP	Any application calls UTLXe via REST. No SDK, no Dapr, no BizTalk — just HTTP in, HTTP out.

All four paths use the same `.utlx` transformations, the same engine binary, the same proto contract. A transformation tested in one path works identically in all others.

3 Quick Start

This chapter gets you from zero to a running transformation in under fifteen minutes. You will deploy UTLXe from the Azure Marketplace, upload a transformation, test it, and send your first real message.

3.1 What You Get

When you deploy UTLXe from the Azure Marketplace, Azure creates a Container App running the UTLXe production engine. The container exposes two ports:

- **Port 8085** — the data plane. Your applications send messages here for transformation. This port is exposed via the Container App ingress.
- **Port 8081** — the admin and health port. You manage transformations here. This port stays internal to your VNet.

The container starts empty — no transformations are loaded. You deploy them via the Admin API.

3.2 Deploy from the Azure Marketplace

1. Open the Azure Portal and search for "UTLXe" in the Marketplace.
2. Select your plan:
   - **Starter** (2 GB) — suitable for messages up to 50 KB, development and light production workloads.
   - **Professional** (4 GB) — suitable for messages up to 200 KB, production workloads with higher throughput.
3. Configure the deployment:
   - Resource group and region.
   - Container App name.
   - Persistent storage toggle — enable this if you want transformations to survive container restarts.
4. Click **Create**. The deployment takes approximately two minutes.

Once complete, note the Container App's internal IP (for the admin port) and the ingress URL (for the data plane).

3.3 Set the Admin Key

Before you can manage transformations, set the admin API key. This key protects the management endpoints from unauthorized access.

```
az containerapp secret set \
  -n utlxe -g myResourceGroup \
  --secrets admin-key=my-secret-key-here

az containerapp update \
  -n utlxe -g myResourceGroup \
  --set-env-vars UTLXE_ADMIN_KEY=secretref:admin-key
```

3.4 Verify the Deployment

Check that the container is running:

```
curl -s http://<internal-ip>:8081/health
```

```
{"status": "UP", "transformations": 0, "ready": false}
```

The engine is alive but has no transformations yet. The `ready: false` flag tells Kubernetes not to route traffic to this container.

3.5 Write Your First Transformation

Create a file called `hello.utlx`:

```
%utlx 1.0
input json
output json
---
{
  greeting: concat("Hello, ", $input.name, "!"),
  timestamp: now()
}
```

This transformation takes a JSON object with a `name` field and produces a greeting with a timestamp.

3.6 Upload It

```
curl -X POST \
  -H "X-Admin-Key: my-secret-key-here" \
  -F "source=@hello.utlx" \
  http://<internal-ip>:8081/admin/transformations/hello
```

```
{
  "status": "deployed",
  "name": "hello",
  "strategy": "COMPILED",
  "config": "defaults",
  "compiled_in_ms": 48
}
```

The transformation was compiled in 48 milliseconds and is ready to process messages. Check the health endpoint again:

```
{"status": "UP", "transformations": 1, "ready": true}
```

Now `ready: true` — the container will accept traffic.

3.7 Test It

Before routing real traffic, verify the transformation works with sample input:

```
curl -X POST \
  -H "X-Admin-Key: my-secret-key-here" \
  -H "Content-Type: application/json" \
  -d '{"name": "World"}' \
  http://<internal-ip>:8081/admin/transformations/hello/test
```

```
{
  "status": "ok",
  "output": {
    "greeting": "Hello, World!",
    "timestamp": "2026-05-05T14:30:00Z"
```

```
  },
  "duration_ms": 2
}
```

The test endpoint executes the transformation but does not count the call in Prometheus metrics. It is a safe way to verify before going live.

3.8 Send a Real Message

Now send a message through the data plane:

```
curl -X POST \
  -H "Content-Type: application/json" \
  -d '{"name": "Azure"}' \
  http://<ingress-url>:8085/api/transform/hello
```

```
{
  "greeting": "Hello, Azure!",
  "timestamp": "2026-05-05T14:30:05Z"
}
```

That is it. The transformation was compiled once at upload time. Every subsequent message executes the compiled version — typically one to five milliseconds per message.

3.9 What Just Happened

The following sequence describes the complete flow:

1. You uploaded `hello.utlx` to the Admin API on port 8081.
2. UTLXe parsed and compiled the transformation into an optimized in-memory representation.
3. The compiled transformation was registered in the engine and written to `/utlxe/data/transformations/hello/`.
4. The health endpoint switched to `ready: true`.
5. Your client sent a JSON message to the data plane on port 8085.
6. UTLXe looked up the compiled `hello` transformation, executed it, and returned the result.

The Admin API and the data plane run simultaneously — there is no "deployment mode" or downtime window.

3.10 Next Steps

- **Chapter 2** explains how to write more complex transformations — conditionals, array operations, multi-format output.
- **Chapter 3** covers the full Admin API — bundles, schemas, testing, and operational management.
- **Chapter 4** shows how to connect UTLXe to Azure Service Bus and Event Hub via Dapr.

4 Writing Transformations

This chapter teaches enough UTL-X to write production transformations. It is not the complete language reference — for that, see the companion book *UTL-X: One Language, All Formats*. Here you learn the patterns that cover ninety percent of real-world integration scenarios.

4.1 The .utlx File Structure

Every transformation has two parts: a header that declares the input and output formats, and a body that defines the transformation logic. They are separated by `---`.

```
%utlx 1.0
input json
output json
---
{
  orderId: $input.id,
  customer: $input.buyer.name,
  total: $input.amount
}
```

The header tells UTLXe how to parse the incoming message and how to serialize the output. The body is an expression that produces the output from the input.

4.2 Property Access

Access input fields with dot notation. The special variable `$input` refers to the incoming message.

```
$input.customer.name          // nested property
$input.items[0].price         // array index (0-based)
$input.@id                    // XML attribute
$input.order.item[2].@qty     // attribute on indexed element
```

If a property does not exist, the result is `null` — no error is thrown.

4.3 Object Construction

Curly braces create new objects. Each line is a key-value pair.

```
{
  orderId: $input.id,
  customer: $input.buyer.name,
  total: $input.quantity * $input.unitPrice
}
```

The spread operator copies all properties from an existing object:

```
{
  ...$input,
  processed: true,
  processedAt: now()
}
```

This produces the original input with two additional fields.

4.4 Let Bindings

Use `let` to name intermediate values. This avoids repetition and makes transformations readable.

```
{
  let subtotal = reduce($input.items, 0, (acc, i) -> acc + i.price)
  let tax = subtotal * 0.21
  let shipping = if (subtotal > 100) 0 else 9.95

  subtotal: subtotal,
  tax: round(tax, 2),
  shipping: shipping,
  total: round(subtotal + tax + shipping, 2)
}
```

4.5 Conditionals

The `if` expression chooses between two values:

```
if ($input.total > 1000) "premium" else "standard"
```

For multiple branches, use `match`:

```
match $input.status {
  "A" -> "Active",
  "I" -> "Inactive",
  "S" -> "Suspended",
  _ -> "Unknown"
}
```

The underscore `_` is the catch-all pattern.

4.6 Array Operations

Arrays are transformed with higher-order functions. The three most common are `map`, `filter`, and `reduce`.

map — transform each element:

```
map($input.items, (item) -> {
  name: item.product,
  total: item.quantity * item.unitPrice
})
```

filter — keep elements matching a condition:

```
filter($input.items, (item) -> item.price > 100)
```

reduce — aggregate to a single value:

```
reduce($input.items, 0, (sum, item) -> sum + item.price)
```

These compose naturally:

```
$input.orders
  | filter(_, (o) -> o.status == "active")
  | map(_, (o) -> {id: o.id, total: o.amount})
  | sortBy(_, (o) -> o.total)
```

The pipe operator `|` passes the result of each step to the next. The underscore `_` is a placeholder for the piped value.

4.7 Common Standard Library Functions

UTL-X has over 650 standard library functions. The ones you will use most often:

4.7.1 Strings

`concat(a, b, c)`	Concatenate strings
`upperCase(s)` / `lowerCase(s)`	Case conversion
`trim(s)`	Remove leading and trailing whitespace
`replace(s, old, new)`	Replace substring
`contains(s, sub)`	Check if string contains substring
`substring(s, start, end)`	Extract substring
`length(s)`	String length
`split(s, delimiter)`	Split string into array
`startsWith(s, prefix)`	Check prefix

4.7.2 Arrays

`map(arr, fn)`	Transform each element
`filter(arr, fn)`	Keep matching elements
`reduce(arr, init, fn)`	Aggregate to single value
`find(arr, fn)`	First matching element
`sortBy(arr, fn)`	Sort by key
`groupBy(arr, fn)`	Group by key
`flatMap(arr, fn)`	Map and flatten
`size(arr)`	Array length
`first(arr)` / `last(arr)`	First or last element

4.7.3 Math and Type

`round(n, decimals)`	Round to decimal places
`floor(n)` / `ceil(n)`	Floor or ceiling
`abs(n)` / `min(a, b)` / `max(a, b)`	Absolute value, min, max
`toString(v)` / `toNumber(v)`	Type conversion
`isNull(v)`	Null check
`now()`	Current timestamp
`formatDate(d, pattern)`	Format a date

4.8 Multi-Format Transformations

The header declares formats. The body works the same regardless of whether the input is JSON, XML, CSV, or YAML — UTL-X operates on the Universal Data Model (UDM), which is format-agnostic.

JSON to XML:

```
%utlx 1.0
input json
output xml
---
{
  Invoice: {
    ID: $input.orderId,
    Amount: $input.total,
    Currency: $input.currency
  }
}
```

The XML serializer produces `<Invoice><ID>12345</ID><Amount>250.0</Amount>...</Invoice>` from this object. You write objects — the format system handles serialization.

XML to JSON:

```
%utlx 1.0
input xml
output json
---
{
  orderId: $input.Invoice.ID,
  total: toNumber($input.Invoice.Amount),
  currency: $input.Invoice.Currency
}
```

CSV to JSON:

```
%utlx 1.0
input csv {header: true}
output json
---
map($input, (row) -> {
  name: row.Name,
  email: lowerCase(row.Email),
  active: row.Status == "A"
})
```

4.9 User-Defined Functions

Define reusable logic with `function`:

```
function discount(amount, tier) {
  match tier {
    "gold" -> amount * 0.10,
    "silver" -> amount * 0.05,
```

```
    _ -> 0
  }
}

{
  let disc = discount($input.total, $input.customerTier)
  orderId: $input.id,
  subtotal: $input.total,
  discount: round(disc, 2),
  total: round($input.total - disc, 2)
}
```

Functions are defined before the main expression. They can call other functions and use all standard library functions.

4.10 Worked Example: Invoice Transformation

A complete transformation that converts a Dynamics 365 Business Central order into a simplified UBL-like invoice:

```
%utlx 1.0
input json
output xml
---
{
  let order = $input
  let subtotal = reduce(order.lines, 0,
    (sum, line) -> sum + line.quantity * line.unitPrice)
  let tax = subtotal * 0.21

  Invoice: {
    ID: concat("INV-", order.orderNumber),
    IssueDate: formatDate(now(), "yyyy-MM-dd"),
    CustomerName: order.customer.name,
    Currency: order.currency,
    InvoiceLines: {
      InvoiceLine: map(order.lines, (line) -> {
        ID: line.lineNumber,
        Quantity: line.quantity,
        UnitPrice: line.unitPrice,
        LineTotal: round(line.quantity * line.unitPrice, 2)
      })
    },
    Subtotal: round(subtotal, 2),
    Tax: round(tax, 2),
    Total: round(subtotal + tax, 2)
  }
}
```

The XML serializer produces the UBL structure from this object. You write objects with property names that become element names — the format system handles the rest.

This transformation demonstrates property access, array iteration with `map`, aggregation with `reduce`, string concatenation, date formatting, and arithmetic.

4.11 Where to Learn More

- The complete language specification, including pattern matching, recursive functions, and the full operator table, is in the companion book *UTL-X: One Language, All Formats*.
- The standard library reference (all 650+ functions with examples) is in chapters 16 and 17 of the companion book.
- Example transformations are available in the project repository under `examples/`.

5 The Admin API

The Admin API is the primary interface for deploying and operating UTLXe on Azure. It runs on port 8081 alongside the health and metrics endpoints. This chapter covers every operation you need for day-to-day management.

5.1 Architecture: Two Ports, Two Purposes

UTLXe exposes two HTTP servers on separate ports:

Port	Purpose	Access
8081	Admin API + health + metrics	Internal to VNet — not exposed via ingress
8085	Data plane (message processing)	Exposed via Container App ingress

This separation allows network isolation. The data plane is accessible to client applications and Dapr sidecars. The admin port is restricted to operators and CI/CD pipelines within the VNet.

Both ports are always active. There is no mode switch — you can upload transformations while the data plane processes messages.

5.2 The Admin Web UI (Optional)

UTLXe ships with an optional web UI — a separate lightweight container (nginx, 5MB) in the same pod. It provides a browser-based interface for all Admin API operations.

Container	Port	Purpose
utlxe	8081 + 8085	Engine — admin API + data plane
dapr	3500	Sidecar — localhost only
utlxe-ui	8088 (default)	Web UI — browser access, proxies `/admin/*` to UTLXe

Open the UI in your browser at the Container App's external URL. The UI provides:

- **Dashboard** — all transformations at a glance with message count, error rate, sync status
- **Transformation detail** — source code view, test panel (run with sample input), error history
- **Upload** — paste `.utlx` source directly or upload a `.zip` / `.utlar` bundle
- **Messaging** — configure queues, topics, Event Hubs per transformation, then sync to Dapr
- **Logs** — view recent log entries, change log level to DEBUG with auto-revert
- **Schemas** — upload, list, delete shared validation schemas
- **Sync overview** — Dapr integration status, loaded components, sync all drafts

In locked mode (production), the UI shows a read-only view. Upload and delete buttons are disabled. Operational actions (pause, resume, test, log level, validation override) remain available.

The UI is **optional**. Customers who manage UTLXe via CI/CD and scripts can omit the `utlxe-ui` container entirely. All functionality is available via the REST API — the UI calls the same endpoints.

All ports are configurable via environment variables (`UI_PORT`, `ADMIN_PORT`). The UI makes zero changes to UTLXe — it is a pure API consumer.

5.3 Authentication

Every request to `/admin/*` endpoints must include the `X-Admin-Key` header:

```
curl -H "X-Admin-Key: my-secret-key-here" \
  http://<internal-ip>:8081/admin/transformations
```

The key is set via the `UTLXE_ADMIN_KEY` environment variable. If this variable is not set, all admin endpoints return 403 — the API is locked by default.

The health endpoints (`/health`, `/metrics`) do not require authentication. They are read-only and needed by Kubernetes probes and Prometheus.

5.4 Uploading Transformations

5.4.1 Single .utlx File

The simplest deployment: upload just the `.utlx` source file. UTLXe applies sensible defaults (COMPILED strategy, auto-detect format).

```
curl -X POST \
  -H "X-Admin-Key: $KEY" \
  -F "source=@invoice-to-ubl.utlx" \
  http://<admin>:8081/admin/transformations/invoice-to-ubl
```

Response:

```
{
  "status": "deployed",
  "name": "invoice-to-ubl",
  "strategy": "COMPILED",
  "config": "defaults",
  "compiled_in_ms": 48
}
```

5.4.2 With Configuration

For explicit control over strategy and validation, include a `transform.yaml`:

```
curl -X POST \
  -H "X-Admin-Key: $KEY" \
  -F "source=@invoice-to-ubl.utlx" \
  -F "config=@transform.yaml" \
  http://<admin>:8081/admin/transformations/invoice-to-ubl
```

A typical `transform.yaml`:

```
strategy: COMPILED
validationPolicy: strict
inputs:
  - name: input
    schema: order.json
maxConcurrent: 4
```

5.4.3 ZIP Bundle

For batch deployment, upload a ZIP containing all transformations and schemas at once. This replaces the entire bundle atomically.

```
curl -X POST \
  -H "X-Admin-Key: $KEY" \
```

```
  -F "file=@bundle.zip" \
  http://<admin>:8081/admin/bundle
```

The ZIP follows this structure:

```
bundle.zip
  schemas/
    order.xsd
    invoice.json
  transformations/
    invoice-to-ubl/
      invoice-to-ubl.utlx
    order-enrichment/
      order-enrichment.utlx
```

The `schemas/` directory is optional. Each transformation directory requires only the `.utlx` source file. A `transform.yaml` can be added alongside the `.utlx` to override defaults (strategy, validation policy, output binding) — but it is not required.

5.5 Managing Schemas

Schemas are shared resources, stored separately from transformations. A single schema like `order.xsd` can be referenced by multiple transformations.

Upload a schema:

```
curl -X POST \
  -H "X-Admin-Key: $KEY" \
  -F "file=@order.xsd" \
  http://<admin>:8081/admin/schemas/order.xsd
```

List all schemas:

```
curl -H "X-Admin-Key: $KEY" \
  http://<admin>:8081/admin/schemas
```

```
{
  "schemas": [
    {"filename": "order.xsd", "size_bytes": 4820, "uploaded_at": "2026-05-05T14:29:00Z"},
    {"filename": "invoice.json", "size_bytes": 2340, "uploaded_at":
"2026-05-05T14:29:05Z"}
  ]
}
```

Updating a schema does not recompile transformations — schemas are resolved at validation time, not compile time. The new schema takes effect on the next message.

5.6 Testing Before Go-Live

The test endpoint is the most important step after uploading a transformation. It executes the transformation with sample input and returns the result without affecting metrics.

```
curl -X POST \
  -H "X-Admin-Key: $KEY" \
  -H "Content-Type: application/json" \
  -d '{"orderId": "12345", "amount": 250.00, "currency": "EUR"}' \
  http://<admin>:8081/admin/transformations/invoice-to-ubl/test
```

On success:

```
{
  "status": "ok",
  "output": {"Invoice": {"ID": "INV-12345", "Amount": 250.0, "Currency": "EUR"}},
  "duration_ms": 3
}
```

On failure:

```
{
  "status": "error",
  "error": "Null reference: $input.customer.address",
  "line": 14,
  "column": 22
}
```

Always test before routing real traffic. The deployment workflow should be: upload, test, then allow traffic.

5.7 Listing and Inspecting

List all deployed transformations:

```
curl -H "X-Admin-Key: $KEY" \
  http://<admin>:8081/admin/transformations
```

```
{
  "transformations": [
    {
      "name": "invoice-to-ubl",
      "strategy": "COMPILED",
      "status": "ready",
      "config": "explicit",
      "deployed_at": "2026-05-05T14:30:00Z",
      "messages_processed": 12345,
      "avg_transform_ms": 2.3
    }
  ]
}
```

Get details for a specific transformation, including the source code:

```
curl -H "X-Admin-Key: $KEY" \
  http://<admin>:8081/admin/transformations/invoice-to-ubl
```

5.8 Updating Transformations

Upload the same name again to replace a transformation. The update is atomic:

1. The new source is compiled.
2. If compilation fails, the upload is rejected — the running transformation is untouched.
3. If compilation succeeds, the new version replaces the old one in the registry.
4. In-flight messages on the old version complete normally.
5. New messages use the new version immediately.

There is no downtime, no mode switch, and no restart.

5.9 Deleting

Remove a single transformation:

```
curl -X DELETE \
  -H "X-Admin-Key: $KEY" \
  http://<admin>:8081/admin/transformations/invoice-to-ubl
```

Remove everything and start fresh:

```
curl -X DELETE \
  -H "X-Admin-Key: $KEY" \
  http://<admin>:8081/admin/bundle
```

5.10 Exporting the Bundle

Download the current state as a ZIP file:

```
curl -H "X-Admin-Key: $KEY" \
  http://<admin>:8081/admin/bundle -o bundle-backup.zip
```

This is useful for version control, backup, or migrating transformations to another container. The exported ZIP has the same format as the upload — you can re-import it with `POST /admin/bundle`.

5.11 Data Plane Discovery

Client applications can discover available transformations without an admin key. This endpoint runs on the data plane port (8085):

```
curl http://<ingress>:8085/api/transformations
```

```
{
  "transformations": [
    {"name": "invoice-to-ubl", "status": "ready", "input": "json", "output": "xml"},
    {"name": "order-enrichment", "status": "ready", "input": "json", "output": "json"}
  ]
}
```

This makes the data plane self-describing. A client connecting for the first time can discover what transformations are available without out-of-band knowledge.

5.12 Engine Info

Get basic operational information:

```
curl -H "X-Admin-Key: $KEY" \
  http://<admin>:8081/admin/info
```

```
{
  "version": "1.0.1",
  "uptime_seconds": 86400,
  "mode": "http",
  "workers": 4,
  "heap_max_mb": 1536,
  "data_dir": "/utlxe/data",
  "persistence": "volume-backed",
  "admin_key_set": true,
  "transformations": 3,
  "schemas": 2,
  "ready": true
}
```

5.13 Two Deployment Workflows

The Admin API supports two workflows. Use whichever fits your process.

Batch workflow — assemble a ZIP in your CI/CD pipeline and deploy in one call:

```
curl -X POST -H "X-Admin-Key: $KEY" \
  -F "file=@bundle.zip" \
  http://<admin>:8081/admin/bundle
```

Incremental workflow — upload resources one at a time:

```
# Upload schemas first
curl -X POST -H "X-Admin-Key: $KEY" -F "file=@order.xsd" .../admin/schemas/order.xsd

# Then transformations
curl -X POST -H "X-Admin-Key: $KEY" -F "source=@invoice.utlx" .../admin/transformations/
invoice

# Export the assembled bundle for version control
curl -H "X-Admin-Key: $KEY" .../admin/bundle -o my-bundle.zip
```

Both workflows produce the same result. You can start with the incremental workflow during development, then switch to batch for production CI/CD.

6 Connecting to Azure Services

This chapter walks through every step of connecting UTLXe to Azure Service Bus and Event Hub. Each section is a complete, self-contained walkthrough — from creating the Azure resources to sending the first message through UTLXe.

6.1 Prerequisites

Before you begin, you need:

- An Azure subscription.
- The Azure CLI installed (`az` command). Install from `https://docs.microsoft.com/en-us/cli/azure/install-azure-cli`.
- A deployed UTLXe Container App (see Chapter 2: Quick Start).
- The UTLXe admin key (`UTLXE_ADMIN_KEY`) configured on the container.

All commands in this chapter use the Azure CLI. Where relevant, the equivalent Azure Portal steps are noted.

6.2 Walkthrough: Service Bus Queue to Queue

This walkthrough connects an Azure Service Bus input queue to UTLXe, transforms messages, and sends results to an output queue.

6.2.1 Step 1: Create the Service Bus Namespace and Queues

```
# Variables — adjust to your environment
RESOURCE_GROUP="myResourceGroup"
LOCATION="westeurope"
SB_NAMESPACE="sb-utlxe-demo"

# Create the Service Bus namespace
az servicebus namespace create \
  --name $SB_NAMESPACE \
  --resource-group $RESOURCE_GROUP \
  --location $LOCATION \
  --sku Standard

# Create the input queue
az servicebus queue create \
  --name incoming-orders \
  --namespace-name $SB_NAMESPACE \
  --resource-group $RESOURCE_GROUP

# Create the output queue
az servicebus queue create \
  --name processed-orders \
  --namespace-name $SB_NAMESPACE \
  --resource-group $RESOURCE_GROUP
```

Portal alternative: In the Azure Portal, go to "Create a resource" > "Service Bus" > fill in namespace name, resource group, region, and Standard tier. Then open the namespace and click "+ Queue" twice to create `incoming-orders` and `processed-orders`.

6.2.2 Step 2: Get the Connection String

```
# Get the connection string for the Dapr component
az servicebus namespace authorization-rule keys list \
```

```
  --name RootManageSharedAccessKey \
  --namespace-name $SB_NAMESPACE \
  --resource-group $RESOURCE_GROUP \
  --query primaryConnectionString -o tsv
```

Copy the connection string. You will store it as a secret in the Container App environment.

6.2.3 Step 3: Store the Connection String as a Secret

```
CONTAINER_APP="utlxe"
CONTAINER_ENV="my-container-env"

# Store as a secret in the Container App
az containerapp secret set \
  --name $CONTAINER_APP \
  --resource-group $RESOURCE_GROUP \
  --secrets servicebus-connection="<paste connection string>"
```

Portal alternative: Open the Container App > "Secrets" > "+ Add" > name: `servicebus-connection`, value: paste the connection string.

6.2.4 Step 4: Configure the Dapr Input Component

Enable Dapr on the Container App and add the input binding component:

```
# Enable Dapr on the Container App (if not already enabled)
az containerapp dapr enable \
  --name $CONTAINER_APP \
  --resource-group $RESOURCE_GROUP \
  --dapr-app-id utlxe \
  --dapr-app-port 8085

# Create the Dapr component for the input queue
az containerapp env dapr-component set \
  --name $CONTAINER_ENV \
  --resource-group $RESOURCE_GROUP \
  --dapr-component-name orders-in \
  --yaml dapr-input.yaml
```

Create `dapr-input.yaml`:

```
componentType: bindings.azure.servicebusqueues
version: v1
metadata:
  - name: connectionString
    secretRef: servicebus-connection
  - name: queueName
    value: "incoming-orders"
scopes:
  - utlxe
```

Portal alternative: Open the Container App Environment > "Dapr components" > "+ Add" > component name: `orders-in`, type: `bindings.azure.servicebusqueues`. Add metadata: `connectionString` (secret reference: `servicebus-connection`), `queueName`: `incoming-orders`. Scope to `utlxe`.

6.2.5 Step 5: Configure the Dapr Output Component

```
az containerapp env dapr-component set \
  --name $CONTAINER_ENV \
  --resource-group $RESOURCE_GROUP \
  --dapr-component-name orders-out \
  --yaml dapr-output.yaml
```

Create `dapr-output.yaml`:

```
componentType: bindings.azure.servicebusqueues
version: v1
metadata:
  - name: connectionString
    secretRef: servicebus-connection
  - name: queueName
    value: "processed-orders"
scopes:
  - utlxe
```

6.2.6 Step 6: Upload the Transformation

Create `orders-in.utlx` — the transformation name must match the Dapr input component name (`orders-in`):

```
%utlx 1.0
input json
output json
---
{
  processedOrderId: concat("PROC-", $input.orderId),
  customer: upperCase($input.customerName),
  total: round($input.quantity * $input.unitPrice, 2),
  processedAt: now()
}
```

Upload it with the output binding configured:

```
# Upload the transformation
curl -X POST \
  -H "X-Admin-Key: $ADMIN_KEY" \
  -F "source=@orders-in.utlx" \
  http://<admin-ip>:8081/admin/transformations/orders-in

# Optionally set the output binding via config
curl -X POST \
  -H "X-Admin-Key: $ADMIN_KEY" \
  -H "Content-Type: application/json" \
  -d '{"outputBinding": "orders-out"}' \
  http://<admin-ip>:8081/admin/transformations/orders-in/config
```

6.2.7 Step 7: Test the Transformation

Before sending real messages, test with sample input:

```
curl -X POST \
  -H "X-Admin-Key: $ADMIN_KEY" \
  -H "Content-Type: application/json" \
  -d '{"orderId":"ORD-001","customerName":"Acme Corp","quantity":5,"unitPrice":24.50}' \
  http://<admin-ip>:8081/admin/transformations/orders-in/test
```

Expected response:

```
{
  "status": "ok",
  "output": {
    "processedOrderId": "PROC-ORD-001",
    "customer": "ACME CORP",
    "total": 122.5,
    "processedAt": "2026-05-05T14:30:00Z"
  },
  "duration_ms": 2
}
```

6.2.8 Step 8: Send a Message to Service Bus

```
# Send a test message to the input queue
az servicebus queue send \
  --name incoming-orders \
  --namespace-name $SB_NAMESPACE \
  --resource-group $RESOURCE_GROUP \
  --body '{"orderId":"ORD-001","customerName":"Acme Corp","quantity":5,"unitPrice":24.50}'
```

Portal alternative: Open the Service Bus namespace > "incoming-orders" queue > "Service Bus Explorer" > "Send message" > paste the JSON body > click "Send".

6.2.9 Step 9: Check the Output Queue

```
# Peek at the output queue
az servicebus queue peek \
  --name processed-orders \
  --namespace-name $SB_NAMESPACE \
  --resource-group $RESOURCE_GROUP
```

You should see the transformed message:

```
{
  "processedOrderId": "PROC-ORD-001",
  "customer": "ACME CORP",
  "total": 122.5,
  "processedAt": "2026-05-05T14:30:05Z"
}
```

The complete flow is now working: Service Bus input queue → Dapr → UTLXe → Dapr → Service Bus output queue.

6.2.10 What Happens on Failure

If the transformation fails (for example, a required field is missing), UTLXe returns HTTP 500 to Dapr. Dapr does not acknowledge the message on Service Bus. Service Bus retries delivery based on the queue's retry policy. After the maximum number of retries, the message moves to the dead-letter queue.

Check recent errors:

```
curl -H "X-Admin-Key: $ADMIN_KEY" \
  http://<admin-ip>:8081/admin/transformations/orders-in/errors
```

6.3 Walkthrough: Event Hub

Event Hub uses the same pattern with a different Dapr component type. Only the differences from the Service Bus walkthrough are shown.

6.3.1 Create the Event Hub

```
EH_NAMESPACE="eh-utlxe-demo"

az eventhubs namespace create \
  --name $EH_NAMESPACE \
  --resource-group $RESOURCE_GROUP \
  --location $LOCATION \
  --sku Standard

az eventhubs eventhub create \
  --name incoming-events \
  --namespace-name $EH_NAMESPACE \
  --resource-group $RESOURCE_GROUP \
  --partition-count 4

# Storage account for checkpointing
az storage account create \
  --name stutlxecheckpoint \
  --resource-group $RESOURCE_GROUP \
  --location $LOCATION \
  --sku Standard_LRS

az storage container create \
  --name checkpoints \
  --account-name stutlxecheckpoint
```

6.3.2 Dapr Component

Create `dapr-eventhub-input.yaml`:

```
componentType: bindings.azure.eventhubs
version: v1
metadata:
  - name: connectionString
    secretRef: eventhub-connection
  - name: consumerGroup
    value: "utlxe-consumer"
  - name: storageAccountName
    value: "stutlxecheckpoint"
```

```
  - name: storageContainerName
    value: "checkpoints"
  - name: storageAccountKey
    secretRef: storage-account-key
scopes:
  - utlxe
```

The transformation is identical to the Service Bus example — UTLXe does not know or care whether the message came from Service Bus or Event Hub.

6.4 Direct HTTP (No Dapr)

For synchronous request/response patterns, clients call UTLXe directly on port 8085. No Dapr configuration is needed.

```
curl -X POST \
  -H "Content-Type: application/json" \
  -d '{"orderId":"ORD-001","customerName":"Acme Corp","quantity":5,"unitPrice":24.50}' \
  http://<ingress-url>:8085/api/transform/orders-in
```

The response is the transformed message, returned synchronously. This is suitable for:

- API gateway integration — transform requests or responses inline.
- Batch processing — send messages from a script or pipeline.
- Testing and development.

No Dapr component, no Service Bus, no queue configuration. Just HTTP in, HTTP out.

6.5 Worked Example: Dynamics 365 to Peppol

A complete end-to-end flow for European e-invoicing:

1. **Dynamics 365 Business Central** publishes an invoice as OData JSON to a Service Bus queue (using D365's built-in Service Bus integration).
2. **Dapr** delivers the message to UTLXe.
3. **UTLXe** runs the `invoice-to-ubl` transformation — converting D365 JSON to UBL 2.1 XML with Peppol BIS 3.0 compliance.
4. **Dapr** publishes the UBL XML to an output Service Bus topic.
5. A **Peppol Access Point** subscribes to the topic and delivers the invoice via AS4.

The transformation, Dapr components, and Service Bus resources are configured following the same steps shown in this chapter. The only difference is the `.utlx` content — which maps D365 fields to UBL elements.

6.6 Bindings vs. Pub/Sub: Which Pattern to Use

Dapr offers two building blocks for messaging. Understanding the difference is essential for choosing the right pattern.

6.6.1 Bindings (Queues — Point-to-Point)

```
Producer → Queue → ONE Consumer
```

- One Dapr component per queue. Static — you create the component YAML, Dapr connects.
- The message is consumed exactly once, then removed from the queue.
- Use case: "Process this order" — exactly one handler.
- Dapr component type: `bindings.azure.servicebusqueues`

- UTLXe receives messages via `POST /{binding-name}` on port 8085.

6.6.2 Pub/Sub (Topics — Fan-out)

```
Producer → Topic → Subscription A → Consumer A
                 → Subscription B → Consumer B
```

- **One** Dapr component per Service Bus namespace (not per topic). Dynamic — UTLXe controls which topics it subscribes to.
- Each subscription gets its own copy of every message.
- Use case: "Notify everyone about this order" — fan-out to multiple systems.
- Dapr component type: `pubsub.azure.servicebus.topics`
- UTLXe receives messages via `POST /pubsub/{transformation-name}` as CloudEvents.

6.6.3 When to Use Which

Criterion	Binding (Queue)	Pub/Sub (Topic)
Consumers	One	Many (fan-out)
Dapr components	One per queue	One per namespace
Dynamic subscriptions	No — YAML per queue	Yes — UTLXe declares via `/dapr/subscribe`
Message format	Raw payload	CloudEvents envelope (UTLXe unwraps automatically)
Recommended for	Simple queue-to-queue	Multi-subscriber, flexible routing

Both patterns can be mixed: input from a queue, output to a topic (or vice versa).

6.6.4 Configuring Pub/Sub via the Admin API

Instead of creating Dapr component YAML manually, configure messaging through the Admin API:

```
# Upload transformation
curl -X POST -H "X-Admin-Key: $ADMIN_KEY" \
  -d @invoice-normalize.utlx \
  http://<admin>:8081/admin/transformations/invoice-normalize

# Set messaging: topic input, topic output
curl -X POST -H "X-Admin-Key: $ADMIN_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "input": {"topic": "raw-invoices", "subscription": "utlxe"},
    "output": {"topic": "normalized-invoices"}
  }' \
  http://<admin>:8081/admin/transformations/invoice-normalize/messaging

# Test via HTTP first (no Dapr needed)
curl -X POST -H "X-Admin-Key: $ADMIN_KEY" \
  -d '{"invoiceId": "INV-001"}' \
  http://<admin>:8081/admin/transformations/invoice-normalize/test

# When ready, sync to Dapr (creates component YAML, activates subscription)
curl -X POST -H "X-Admin-Key: $ADMIN_KEY" \
  http://<admin>:8081/admin/transformations/invoice-normalize/sync
```

The messaging field name is the discriminator:

Field	Azure Service	Dapr Component
`queue: "name"`	Service Bus Queue	`bindings.azure.servicebusqueues`
`topic: "name"`	Service Bus Topic	`pubsub.azure.servicebus.topics`
`eventhub: "name"`	Event Hub	`bindings.azure.eventhubs`

Input and output can use different services: `queue` in → `topic` out, `eventhub` in → `queue` out, etc.

6.6.5 Authentication: Managed Identity (Recommended)

For production, use Azure Managed Identity instead of connection strings. No secrets to manage — the container's identity authenticates directly with Service Bus or Event Hub.

Auth method	How
Managed Identity (secretless)	Set `namespaceName` in Dapr component. No `connectionString` needed. Assign `Azure Service Bus Data Sender` + `Azure Service Bus Data Receiver` RBAC roles to the Container App identity.
Connection string	Store as Container App secret. Reference via `secretRef` in Dapr component YAML.

The bundle (`.utlar`) never contains credentials. It declares *what* to connect to (queue/topic names). The *how* (auth) comes from the environment.

6.7 Summary: What You Configure Where

What	Where	How
Service Bus / Event Hub	Azure Portal or CLI	`az servicebus` / `az eventhubs` commands
Connection string or MI	Container App Environment	Secret or Managed Identity RBAC
Dapr binding (queue)	Container App Environment	Dapr component YAML — or auto-generated by UTLXe via sync
Dapr pub/sub (topic)	Container App Environment	Dapr component YAML — or auto-generated by UTLXe via sync
Transformation	UTLXe Admin API or `.utlar`	`POST /admin/transformations/{name}` or CI/CD bundle
Messaging config	UTLXe Admin API or `transform.yaml`	`POST .../messaging` + `POST .../sync`

The Azure resources and Dapr components are configured once. The transformations can be updated at any time without changing the infrastructure. In dev/test (open mode), the Admin API configures everything dynamically. In production (locked mode), the `.utlar` bundle contains all configuration and is deployed via CI/CD.

7 DTAP: From Development to Production

Enterprise deployments follow a promotion path: Development → Test → Acceptance → Production. UTLXe's two-mode architecture (open and locked) maps directly to this lifecycle. This chapter explains how to set up each environment and promote transformations through the stages.

7.1 The Two Modes

What's on disk	Mode	Behavior
Directory structure (no `.utlar`)	**Open** (Dev/Test)	Full Admin API. Upload, edit, delete, test interactively.
`bundle.utlar` file	**Locked** (Acc/Prd)	Admin API read-only. Changes via CI/CD only. Operational endpoints (pause, resume, logs) remain available.

The mode is determined automatically by the presence of `bundle.utlar` on the data volume. No CLI flag, no environment variable — just the file.

7.2 Environment Layout

A typical DTAP setup uses four Azure Container App environments, each with its own Service Bus namespace and UTLXe instance:

Environment	Mode	Service Bus	Persistence	Who deploys
Development	Open	`sb-utlxe-dev`	Azure Files (optional)	Developer via Admin API
Test	Open	`sb-utlxe-tst`	Azure Files	Developer or CI/CD
Acceptance	Locked	`sb-utlxe-acc`	Azure Files + `.utlar`	CI/CD pipeline
Production	Locked	`sb-utlxe-prd`	Azure Files + `.utlar`	CI/CD pipeline (with approval gate)

Each environment connects to its own Service Bus namespace. The same `.utlar` bundle deploys to all locked environments — the bundle contains transformation logic and queue/topic names, but **not** connection strings or credentials.

7.3 Development Workflow (Open Mode)

In development, the operator (or developer) uses the Admin API interactively:

```
# Upload a transformation
curl -X POST -H "X-Admin-Key: $KEY" \
  -d @invoice-normalize.utlx \
  http://utlxe-dev:8081/admin/transformations/invoice-normalize

# Configure messaging (staged as draft)
curl -X POST -H "X-Admin-Key: $KEY" \
  -H "Content-Type: application/json" \
  -d '{"input": {"queue": "raw-invoices"}, "output": {"queue": "normalized"}}' \
  http://utlxe-dev:8081/admin/transformations/invoice-normalize/messaging

# Test with sample data (no queue impact)
curl -X POST -H "X-Admin-Key: $KEY" \
  -d '{"invoiceId": "INV-001", "amount": 100}' \
  http://utlxe-dev:8081/admin/transformations/invoice-normalize/test

# When satisfied, sync to Dapr (messages start flowing)
```

```
curl -X POST -H "X-Admin-Key: $KEY" \
  http://utlxe-dev:8081/admin/transformations/invoice-normalize/sync
```

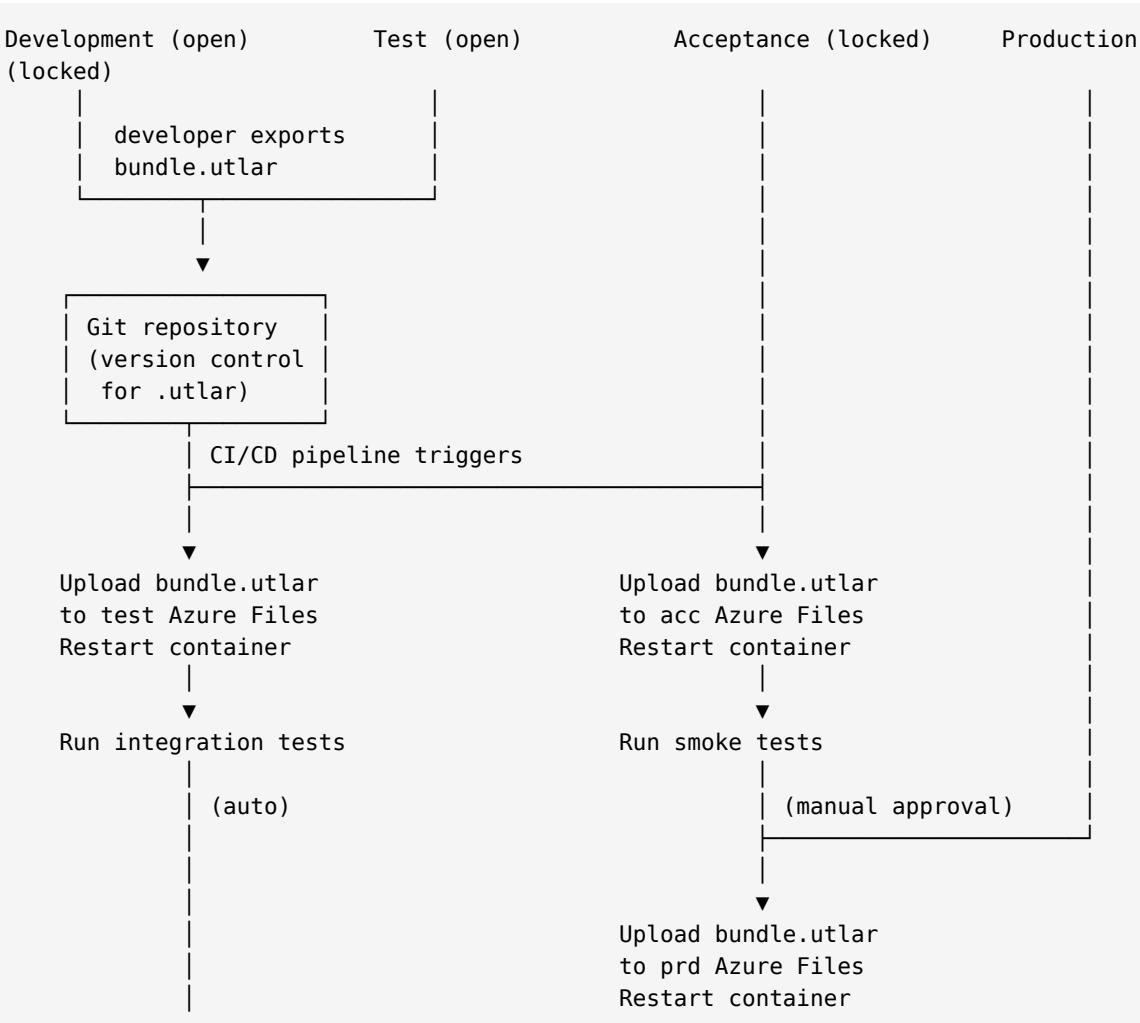
The stage-then-sync model means you can test transformations via HTTP before connecting them to real queues. Sync is the "go live" switch for the development environment.

When the transformation works correctly, export the full bundle:

```
curl -H "X-Admin-Key: $KEY" \
  http://utlxe-dev:8081/admin/bundle -o bundle.utlar
```

This `.utlar` file contains everything: transformations, schemas, transform.yaml with messaging config. It is the artifact that moves through the promotion pipeline.

7.4 Promotion Pipeline



7.5 CI/CD: GitHub Actions Example

```
name: Deploy UTLXe Bundle
on:
  push:
```

```
    paths: ['bundles/**']
    branches: [main]

jobs:
  deploy-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Deploy to Test
        run: |
          az storage file upload \
            --share-name utlxe-data \
            --account-name ${{ secrets.TEST_STORAGE }} \
            --source bundles/bundle.utlar \
            --path bundle.utlar
          az containerapp revision restart \
            -n utlxe -g rg-utlxe-tst

      - name: Wait for ready
        run: |
          for i in $(seq 1 30); do
            STATUS=$(curl -s -o /dev/null -w "%{http_code}" \
              https://utlxe-tst.internal.example.azurecontainerapps.io/health/ready)
            if [ "$STATUS" = "200" ]; then break; fi
            sleep 2
          done

      - name: Run integration tests
        run: ./tests/integration-tests.sh utlxe-tst

  deploy-acceptance:
    needs: deploy-test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Deploy to Acceptance
        run: |
          az storage file upload \
            --share-name utlxe-data \
            --account-name ${{ secrets.ACC_STORAGE }} \
            --source bundles/bundle.utlar \
            --path bundle.utlar
          az containerapp revision restart \
            -n utlxe -g rg-utlxe-acc

  deploy-production:
    needs: deploy-acceptance
    runs-on: ubuntu-latest
    environment: production    # requires manual approval
    steps:
      - uses: actions/checkout@v4
      - name: Deploy to Production
        run: |
```

```
          az storage file upload \
            --share-name utlxe-data \
            --account-name ${{ secrets.PRD_STORAGE }} \
            --source bundles/bundle.utlar \
            --path bundle.utlar
          az containerapp revision restart \
            -n utlxe -g rg-utlxe-prd
```

The key: the **same** `bundle.utlar` file moves from Test → Acceptance → Production. Only the Azure infrastructure (Service Bus namespace, connection strings, managed identity) differs per environment.

7.6 What Differs Per Environment

Aspect	In the bundle (same everywhere)	In the environment (differs per stage)
Transformation logic	`.utlx` source files	—
Queue/topic names	`transform.yaml` messaging config	—
Schemas	`.json` / `.xsd` files	—
Validation policy	`transform.yaml`	—
Service Bus namespace	—	Dapr component YAML / Managed Identity
Connection credentials	—	Container App secrets or Managed Identity RBAC
Admin key	—	`UTLXE_ADMIN_KEY` environment variable
Log level	—	Runtime via Admin API (operational)

This separation means the bundle is environment-agnostic. A transformation that reads from `orders-in` works in dev (where `orders-in` is a queue on `sb-utlxe-dev`) and in production (where `orders-in` is a queue on `sb-utlxe-prd`) — the queue names are the same, the infrastructure behind them differs.

7.7 Same Queue Names, Different Namespaces

The recommended pattern: use the **same** queue and topic names in all environments. Each environment has its own Service Bus namespace:

Queue name	Dev namespace	Acc namespace	Prd namespace
`orders-in`	`sb-utlxe-dev`	`sb-utlxe-acc`	`sb-utlxe-prd`
`orders-out`	`sb-utlxe-dev`	`sb-utlxe-acc`	`sb-utlxe-prd`
`invoices-in`	`sb-utlxe-dev`	`sb-utlxe-acc`	`sb-utlxe-prd`

This way, the bundle's `transform.yaml` says `queue: orders-in` and it works everywhere. The Dapr component (or Managed Identity) points to the correct namespace per environment.

7.8 Operational Capabilities in Locked Mode

Even in production (locked mode), operators have full access to diagnostic and operational endpoints:

Action	How
Pause a transformation	`POST /admin/transformations/{name}/pause`
Resume after fix deployed	`POST /admin/transformations/{name}/resume`

Check errors	`GET /admin/transformations/{name}/errors`
Override validation (incident)	`POST /admin/transformations/{name}/validation`
Switch to DEBUG logging	`POST /admin/log/level` with `{"level":"DEBUG","revert_after_minutes":30}`
View recent logs	`GET /admin/logs?level=ERROR`
Check bundle version	`GET /admin/info` → `bundle_version`, `bundle_checksum`
Export current bundle	`GET /admin/bundle`
Test with sample input	`POST /admin/transformations/{name}/test`

What is **not** available: uploading, deleting, or modifying transformations, schemas, or messaging config. These return 403 `BUNDLE_LOCKED`.

7.9 Rollback

To rollback in production:

1. Deploy the previous version's `bundle.utlar` to Azure Files (from git history or artifact store).
2. Restart the container: `az containerapp revision restart -n utlxe -g rg-utlxe-prd`.
3. UTLXe loads the previous bundle. Done.

The `.utlar` file is the single artifact. Version control it in git alongside the CI/CD pipeline. Every version is a deployable, self-contained unit.

7.10 Infrastructure as Code

All four environments should be defined in Terraform or Bicep. Each environment is identical except for:

- Resource group name and tags
- Service Bus namespace name
- Storage account name
- Container App secrets (admin key, connection strings)
- Managed Identity RBAC role assignments

```
# Example: deploy all four environments from Bicep
az deployment group create -g rg-utlxe-dev -f main.bicep -p env=dev
az deployment group create -g rg-utlxe-tst -f main.bicep -p env=tst
az deployment group create -g rg-utlxe-acc -f main.bicep -p env=acc
az deployment group create -g rg-utlxe-prd -f main.bicep -p env=prd
```

The Bicep template is parameterized by environment. The transformation bundle is **not** part of the infrastructure — it is a separate CI/CD artifact.

8 Infrastructure as Code: Deploying UTLXe with Terraform and Bicep

The Marketplace offering deploys UTLXe via the Azure Portal wizard. But enterprise customers manage infrastructure as code — repeatable, version-controlled, reviewable. This chapter shows how to provision the complete UTLXe stack (Container App, Dapr, Service Bus, storage, networking) using Terraform and Bicep, integrated into CI/CD pipelines.

8.1 What Gets Provisioned

A production UTLXe deployment consists of:

Resource	Purpose
Container App Environment	Hosting environment with VNet integration
Container App (UTLXe)	The engine — runs the UTLXe container image with Dapr sidecar + optional UI container
Azure Files share	Persistent storage for transformations (or `.utlar` in locked mode)
Service Bus namespace	Message queues and/or topics
Dapr components	Binding and pub/sub configuration connecting UTLXe to Service Bus
Managed Identity	Secretless authentication to Service Bus
Log Analytics workspace	Container logs and metrics

8.2 Bicep: Complete Example

This Bicep template provisions everything for one environment. Parameterize `env` to deploy Dev, Test, Acc, and Prd from the same template.

```
@description('Environment name (dev, tst, acc, prd)')
param env string

@description('Azure region')
param location string = resourceGroup().location

@description('UTLXe container image')
param utlxeImage string = 'ghcr.io/grauwen/utlxe:latest'

@description('UTLXe admin API key')
@secure()
param adminKey string

// ── Naming convention ──
var prefix = 'utlxe-${env}'
var sbNamespace = 'sb-${prefix}'
var storageName = 'st${replace(prefix, '-', '')}' // no hyphens in storage names

// ── Log Analytics ──
resource logAnalytics 'Microsoft.OperationalInsights/workspaces@2023-09-01' = {
  name: '${prefix}-logs'
  location: location
  properties: { sku: { name: 'PerGB2018' } }
}

// ── Container App Environment ──
```

```
resource containerEnv 'Microsoft.App/managedEnvironments@2024-03-01' = {
  name: '${prefix}-env'
  location: location
  properties: {
    appLogsConfiguration: {
      destination: 'log-analytics'
      logAnalyticsConfiguration: {
        customerId: logAnalytics.properties.customerId
        sharedKey: logAnalytics.listKeys().primarySharedKey
      }
    }
    daprAIInstrumentationKey: ''
  }
}

// ── Storage Account + File Share ──
resource storageAccount 'Microsoft.Storage/storageAccounts@2023-05-01' = {
  name: storageName
  location: location
  sku: { name: 'Standard_LRS' }
  kind: 'StorageV2'
}

resource fileService 'Microsoft.Storage/storageAccounts/fileServices@2023-05-01' = {
  parent: storageAccount
  name: 'default'
}

resource fileShare 'Microsoft.Storage/storageAccounts/fileServices/shares@2023-05-01' = {
  parent: fileService
  name: 'utlxe-data'
  properties: { shareQuota: 1 }
}

// ── Mount storage in Container App Environment ──
resource envStorage 'Microsoft.App/managedEnvironments/storages@2024-03-01' = {
  parent: containerEnv
  name: 'utlxe-storage'
  properties: {
    azureFile: {
      accountName: storageAccount.name
      accountKey: storageAccount.listKeys().keys[0].value
      shareName: 'utlxe-data'
      accessMode: 'ReadWrite'
    }
  }
}

// ── Service Bus ──
resource serviceBus 'Microsoft.ServiceBus/namespaces@2022-10-01-preview' = {
  name: sbNamespace
  location: location
  sku: { name: 'Standard', tier: 'Standard' }
}
```

```
resource queueOrdersIn 'Microsoft.ServiceBus/namespaces/queues@2022-10-01-preview' = {
  parent: serviceBus
  name: 'orders-in'
  properties: { maxDeliveryCount: 100 }
}

resource queueOrdersOut 'Microsoft.ServiceBus/namespaces/queues@2022-10-01-preview' = {
  parent: serviceBus
  name: 'orders-out'
  properties: { maxDeliveryCount: 100 }
}

// ── Managed Identity for Service Bus ──
resource utlxeApp 'Microsoft.App/containerApps@2024-03-01' = {
  name: prefix
  location: location
  identity: { type: 'SystemAssigned' }
  properties: {
    managedEnvironmentId: containerEnv.id
    configuration: {
      dapr: {
        enabled: true
        appId: 'utlxe'
        appPort: 8085
        appProtocol: 'http'
      }
      ingress: {
        external: true
        targetPort: 8088        // UI container (browser access)
        transport: 'http'
        additionalPortMappings: [
          { targetPort: 8081, exposedPort: 8081, external: false }  // admin API (VNet
only)
          { targetPort: 8085, exposedPort: 8085, external: true }   // data plane
        ]
      }
      secrets: [
        { name: 'admin-key', value: adminKey }
      ]
    }
    template: {
      containers: [
        {
          name: 'utlxe'
          image: utlxeImage
          resources: { cpu: json('1.0'), memory: '2Gi' }
          env: [
            { name: 'UTLXE_ADMIN_KEY', secretRef: 'admin-key' }
          ]
          command: [
            'java', '-jar', '/utlxe/utlxe.jar'
            '--mode', 'http'
            '--admin-port', '8081'
```

```
            '--http-port', '8085'
            '--data-dir', '/utlxe/data'
          ]
          volumeMounts: [
            { volumeName: 'data', mountPath: '/utlxe/data' }
          ]
        }
        // Optional: Admin Web UI container (remove if not needed)
        {
          name: 'utlxe-ui'
          image: 'ghcr.io/grauwen/utlxe-ui:latest'
          resources: { cpu: json('0.25'), memory: '0.5Gi' }
          env: [
            { name: 'UI_PORT', value: '8088' }
            { name: 'ADMIN_PORT', value: '8081' }
          ]
        }
      ]
      scale: { minReplicas: 1, maxReplicas: 3 }
      volumes: [
        { name: 'data', storageName: 'utlxe-storage', storageType: 'AzureFile' }
      ]
    }
  }
}

// ── RBAC: Managed Identity → Service Bus ──
var sbDataSenderRole = '69a216fc-b8fb-44d8-bc22-1f3c2cd27a39'
var sbDataReceiverRole = '4f6d3b9b-027b-4f4c-9142-0e5a2a2247e0'

resource sbSender 'Microsoft.Authorization/roleAssignments@2022-04-01' = {
  name: guid(utlxeApp.id, sbDataSenderRole, serviceBus.id)
  scope: serviceBus
  properties: {
    roleDefinitionId: subscriptionResourceId(
      'Microsoft.Authorization/roleDefinitions', sbDataSenderRole)
    principalId: utlxeApp.identity.principalId
    principalType: 'ServicePrincipal'
  }
}

resource sbReceiver 'Microsoft.Authorization/roleAssignments@2022-04-01' = {
  name: guid(utlxeApp.id, sbDataReceiverRole, serviceBus.id)
  scope: serviceBus
  properties: {
    roleDefinitionId: subscriptionResourceId(
      'Microsoft.Authorization/roleDefinitions', sbDataReceiverRole)
    principalId: utlxeApp.identity.principalId
    principalType: 'ServicePrincipal'
  }
}

// ── Dapr Component: Service Bus binding (Managed Identity) ──
resource daprOrdersIn 'Microsoft.App/managedEnvironments/daprComponents@2024-03-01' = {
```

```
  parent: containerEnv
  name: 'orders-in'
  properties: {
    componentType: 'bindings.azure.servicebusqueues'
    version: 'v1'
    metadata: [
      { name: 'namespaceName', value: '${sbNamespace}.servicebus.windows.net' }
      { name: 'queueName', value: 'orders-in' }
      { name: 'direction', value: 'input, output' }
    ]
    scopes: [ 'utlxe' ]
  }
}

resource daprOrdersOut 'Microsoft.App/managedEnvironments/daprComponents@2024-03-01' = {
  parent: containerEnv
  name: 'orders-out'
  properties: {
    componentType: 'bindings.azure.servicebusqueues'
    version: 'v1'
    metadata: [
      { name: 'namespaceName', value: '${sbNamespace}.servicebus.windows.net' }
      { name: 'queueName', value: 'orders-out' }
      { name: 'direction', value: 'input, output' }
    ]
    scopes: [ 'utlxe' ]
  }
}

// ── Dapr Resiliency (pause/429 circuit breaker) ──
resource daprResiliency 'Microsoft.App/managedEnvironments/daprComponents@2024-03-01' = {
  parent: containerEnv
  name: 'utlxe-resiliency'
  properties: {
    componentType: 'resiliency.azure'
    version: 'v1'
    metadata: []
    scopes: [ 'utlxe' ]
  }
}

// ── Outputs ──
output containerAppFqdn string = utlxeApp.properties.configuration.ingress.fqdn
output serviceBusNamespace string = serviceBus.name
output storageAccount string = storageAccount.name
```

Deploy all four environments:

```
# Dev
az deployment group create -g rg-utlxe-dev \
  -f main.bicep -p env=dev adminKey='dev-key-123'

# Test
az deployment group create -g rg-utlxe-tst \
```

```
  -f main.bicep -p env=tst adminKey='tst-key-456'

# Acceptance
az deployment group create -g rg-utlxe-acc \
  -f main.bicep -p env=acc adminKey='acc-key-789'

# Production
az deployment group create -g rg-utlxe-prd \
  -f main.bicep -p env=prd adminKey='prd-key-secure'
```

8.3 Terraform: Complete Example

The same stack in Terraform, using the AzureRM provider.

```
variable "env" {
  description = "Environment name (dev, tst, acc, prd)"
  type        = string
}

variable "location" {
  description = "Azure region"
  type        = string
  default     = "westeurope"
}

variable "admin_key" {
  description = "UTLXe admin API key"
  type        = string
  sensitive   = true
}

variable "utlxe_image" {
  description = "UTLXe container image"
  type        = string
  default     = "ghcr.io/grauwen/utlxe:latest"
}

locals {
  prefix       = "utlxe-${var.env}"
  sb_namespace = "sb-${local.prefix}"
  storage_name = "st${replace(local.prefix, "-", "")}"
}

resource "azurerm_resource_group" "rg" {
  name     = "rg-${local.prefix}"
  location = var.location
}

# ── Log Analytics ──
resource "azurerm_log_analytics_workspace" "logs" {
  name                = "${local.prefix}-logs"
  location            = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name
  sku                 = "PerGB2018"
```

```
}

# ── Container App Environment ──
resource "azurerm_container_app_environment" "env" {
  name                       = "${local.prefix}-env"
  location                   = azurerm_resource_group.rg.location
  resource_group_name        = azurerm_resource_group.rg.name
  log_analytics_workspace_id = azurerm_log_analytics_workspace.logs.id
}

# ── Storage ──
resource "azurerm_storage_account" "storage" {
  name                     = local.storage_name
  resource_group_name      = azurerm_resource_group.rg.name
  location                 = azurerm_resource_group.rg.location
  account_tier             = "Standard"
  account_replication_type = "LRS"
}

resource "azurerm_storage_share" "data" {
  name               = "utlxe-data"
  storage_account_id = azurerm_storage_account.storage.id
  quota              = 1
}

resource "azurerm_container_app_environment_storage" "data" {
  name                         = "utlxe-storage"
  container_app_environment_id = azurerm_container_app_environment.env.id
  account_name                 = azurerm_storage_account.storage.name
  share_name                   = azurerm_storage_share.data.name
  access_key                   = azurerm_storage_account.storage.primary_access_key
  access_mode                  = "ReadWrite"
}

# ── Service Bus ──
resource "azurerm_servicebus_namespace" "sb" {
  name                = local.sb_namespace
  location            = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name
  sku                 = "Standard"
}

resource "azurerm_servicebus_queue" "orders_in" {
  name                = "orders-in"
  namespace_id        = azurerm_servicebus_namespace.sb.id
  max_delivery_count  = 100
}

resource "azurerm_servicebus_queue" "orders_out" {
  name                = "orders-out"
  namespace_id        = azurerm_servicebus_namespace.sb.id
  max_delivery_count  = 100
}
```

```
# ── Container App ──
resource "azurerm_container_app" "utlxe" {
  name                         = local.prefix
  container_app_environment_id = azurerm_container_app_environment.env.id
  resource_group_name          = azurerm_resource_group.rg.name
  revision_mode                = "Single"

  identity {
    type = "SystemAssigned"
  }

  dapr {
    app_id   = "utlxe"
    app_port = 8085
  }

  ingress {
    external_enabled = true
    target_port      = 8088     # UI container (browser access)
    transport        = "http"
  }

  secret {
    name  = "admin-key"
    value = var.admin_key
  }

  template {
    container {
      name   = "utlxe"
      image  = var.utlxe_image
      cpu    = 1.0
      memory = "2Gi"

      env {
        name        = "UTLXE_ADMIN_KEY"
        secret_name = "admin-key"
      }

      command = [
        "java", "-jar", "/utlxe/utlxe.jar",
        "--mode", "http",
        "--admin-port", "8081",
        "--http-port", "8085",
        "--data-dir", "/utlxe/data",
      ]

      volume_mounts {
        name = "data"
        path = "/utlxe/data"
      }
    }

    # Optional: Admin Web UI container
```

```
    container {
      name   = "utlxe-ui"
      image  = "ghcr.io/grauwen/utlxe-ui:latest"
      cpu    = 0.25
      memory = "0.5Gi"

      env {
        name  = "UI_PORT"
        value = "8088"
      }
      env {
        name  = "ADMIN_PORT"
        value = "8081"
      }
    }

    volume {
      name         = "data"
      storage_name = azurerm_container_app_environment_storage.data.name
      storage_type = "AzureFile"
    }

    min_replicas = 1
    max_replicas = 3
  }
}

# ── RBAC: Managed Identity → Service Bus ──
resource "azurerm_role_assignment" "sb_sender" {
  scope                = azurerm_servicebus_namespace.sb.id
  role_definition_name = "Azure Service Bus Data Sender"
  principal_id         = azurerm_container_app.utlxe.identity[0].principal_id
}

resource "azurerm_role_assignment" "sb_receiver" {
  scope                = azurerm_servicebus_namespace.sb.id
  role_definition_name = "Azure Service Bus Data Receiver"
  principal_id         = azurerm_container_app.utlxe.identity[0].principal_id
}

# ── Deploy bundle.utlar (locked mode) ──
resource "azurerm_storage_share_file" "bundle" {
  count            = fileexists("${path.module}/bundle.utlar") ? 1 : 0
  name             = "bundle.utlar"
  storage_share_id = azurerm_storage_share.data.id
  source           = "${path.module}/bundle.utlar"
}

# ── Outputs ──
output "fqdn" {
  value = azurerm_container_app.utlxe.ingress[0].fqdn
}

output "service_bus" {
```

```
  value = azurerm_servicebus_namespace.sb.name
}
```

Deploy all environments:

```
# Dev (no .utlar → open mode)
terraform workspace select dev
terraform apply -var="env=dev" -var="admin_key=dev-key-123"

# Production (with .utlar → locked mode)
cp artifacts/bundle.utlar .
terraform workspace select prd
terraform apply -var="env=prd" -var="admin_key=prd-key-secure"
```

8.4 CI/CD: Full Pipeline with Infrastructure + Bundle

A complete pipeline manages both layers:

Layer	What changes	How deployed
Infrastructure	Container App, Service Bus, Dapr components, RBAC	Terraform/Bicep — changes rarely
Bundle	Transformations, schemas, messaging config	`.utlar` to Azure Files — changes frequently

8.4.1 GitHub Actions: Infrastructure + Bundle Pipeline

```
name: Full Deployment
on:
  push:
    branches: [main]

jobs:
  # ── Infrastructure (only when infra files change) ──
  infrastructure:
    if: contains(github.event.head_commit.modified, 'infra/')
    runs-on: ubuntu-latest
    strategy:
      matrix:
        env: [tst, acc]    # prd requires manual trigger
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3

      - name: Terraform Apply
        working-directory: infra
        env:
          ARM_CLIENT_ID: ${{ secrets.AZURE_CLIENT_ID }}
          ARM_TENANT_ID: ${{ secrets.AZURE_TENANT_ID }}
          ARM_SUBSCRIPTION_ID: ${{ secrets.AZURE_SUBSCRIPTION_ID }}
        run: |
          terraform init
          terraform workspace select ${{ matrix.env }}
          terraform apply -auto-approve \
            -var="env=${{ matrix.env }}" \
```

```
          -var="admin_key=${{ secrets[format('ADMIN_KEY_{0}', matrix.env)] }}"

  # ── Bundle (when transformation files change) ──
  bundle:
    if: contains(github.event.head_commit.modified, 'transformations/')
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Build bundle
        run: |
          cd transformations
          zip -r ../bundle.utlar manifest.json transformations/ schemas/

      - name: Deploy to Test
        run: |
          az storage file upload \
            --account-name ${{ secrets.TST_STORAGE }} \
            --share-name utlxe-data \
            --source bundle.utlar --path bundle.utlar
          az containerapp revision restart \
            -n utlxe-tst -g rg-utlxe-tst

      - name: Verify Test
        run: |
          sleep 10
          STATUS=$(curl -s -o /dev/null -w "%{http_code}" \
            -H "X-Admin-Key: ${{ secrets.ADMIN_KEY_tst }}" \
            "https://utlxe-tst.${{ secrets.TST_ENV_FQDN }}/health/ready")
          [ "$STATUS" = "200" ] || exit 1

      - name: Deploy to Acceptance
        run: |
          az storage file upload \
            --account-name ${{ secrets.ACC_STORAGE }} \
            --share-name utlxe-data \
            --source bundle.utlar --path bundle.utlar
          az containerapp revision restart \
            -n utlxe-acc -g rg-utlxe-acc

  # ── Production (manual approval) ──
  production:
    needs: bundle
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: actions/checkout@v4
      - name: Deploy to Production
        run: |
          az storage file upload \
            --account-name ${{ secrets.PRD_STORAGE }} \
            --share-name utlxe-data \
            --source bundle.utlar --path bundle.utlar
```

```
          az containerapp revision restart \
            -n utlxe-prd -g rg-utlxe-prd
```

8.4.2 Azure DevOps: Bicep + Bundle Pipeline

```yaml
trigger:
  branches: { include: [main] }

stages:
  - stage: Infrastructure
    condition: |
      contains(variables['Build.SourceVersionMessage'], '[infra]')
    jobs:
      - job: DeployInfra
        strategy:
          matrix:
            tst: { env: tst }
            acc: { env: acc }
        steps:
          - task: AzureCLI@2
            inputs:
              azureSubscription: 'my-service-connection'
              scriptType: bash
              inlineScript: |
                az deployment group create \
                  -g rg-utlxe-$(env) \
                  -f infra/main.bicep \
                  -p env=$(env) adminKey=$(ADMIN_KEY_$(env))

  - stage: Bundle
    condition: always()
    jobs:
      - job: DeployBundle
        steps:
          - script: |
              cd transformations
              zip -r $(Build.ArtifactStagingDirectory)/bundle.utlar \
                manifest.json transformations/ schemas/
            displayName: Build bundle

          - task: AzureCLI@2
            displayName: Deploy to Test
            inputs:
              azureSubscription: 'my-service-connection'
              scriptType: bash
              inlineScript: |
                az storage file upload \
                  --account-name $(TST_STORAGE) \
                  --share-name utlxe-data \
                  --source $(Build.ArtifactStagingDirectory)/bundle.utlar \
                  --path bundle.utlar
                az containerapp revision restart \
                  -n utlxe-tst -g rg-utlxe-tst
```

```yaml
  - stage: Production
    condition: succeeded()
    jobs:
      - deployment: DeployProd
        environment: production    # approval gate
        strategy:
          runOnce:
            deploy:
              steps:
                - task: AzureCLI@2
                  inputs:
                    azureSubscription: 'my-service-connection'
                    scriptType: bash
                    inlineScript: |
                      az storage file upload \
                        --account-name $(PRD_STORAGE) \
                        --share-name utlxe-data \
                        --source $(Pipeline.Workspace)/bundle.utlar \
                        --path bundle.utlar
                      az containerapp revision restart \
                        -n utlxe-prd -g rg-utlxe-prd
```

8.5 Key Decisions

8.5.1 Terraform vs. Bicep

Aspect	Terraform	Bicep
State management	Remote state (Azure Storage backend)	No state — ARM manages it
Multi-cloud	Yes — same tool for AWS, GCP	Azure only
Dapr components	Via `azurerm_container_app_environment_dapr_component`	Via nested resource
Workspaces/DTAP	Terraform workspaces or tfvars	Parameter files per environment
Learning curve	HCL syntax	ARM-like, familiar to Azure teams

Both work. Use what your team already knows.

8.5.2 Bundle Deployment: Azure Files vs Admin API

Method	Azure Files (`.utlar`)	Admin API (`POST /admin/bundle`)
Mode	Locked (production)	Open (dev/test)
Requires restart?	Yes — container restarts to load `.utlar`	No — hot-swap, zero downtime
Immutable?	Yes — Admin API is read-only	No — can be changed at any time
Best for	Acc/Prd — CI/CD deploys artifact	Dev/Tst — interactive development

Production uses Azure Files + `.utlar` (immutable, locked). Development uses the Admin API (dynamic, interactive). The same transformations work in both — the deployment method differs, not the content.

9 CI/CD Integration

Automated deployment ensures that transformations are tested, versioned, and deployed consistently. This chapter shows how to integrate UTLXe with GitHub Actions and Azure DevOps.

9.1 The Deployment Model

The transformation source lives in a git repository. The CI/CD pipeline:

1. Assembles a bundle (ZIP) from the repository contents.
2. Validates the bundle against the running UTLXe instance (dry run).
3. Deploys the bundle.
4. Verifies with test inputs.

No custom Docker image is built. The UTLXe container from the Marketplace stays as-is. Only the transformations change.

9.2 Repository Structure

A recommended layout for the transformation repository:

```
my-transformations/
  schemas/
    order.xsd
    invoice.json
  transformations/
    invoice-to-ubl/
      invoice-to-ubl.utlx
    order-enrichment/
      order-enrichment.utlx
  test-data/
    invoice-to-ubl/
      input.json
      expected-output.xml
  scripts/
    build-bundle.sh
    deploy.sh
```

The `test-data/` directory contains sample inputs and expected outputs for each transformation. The pipeline uses these for post-deployment verification.

9.3 Building the Bundle

A simple script assembles the ZIP:

```
#!/bin/bash
# build-bundle.sh
cd "$(dirname "$0")/.."
zip -r bundle.zip schemas/ transformations/
echo "Bundle created: bundle.zip"
```

9.4 GitHub Actions Workflow

```
name: Deploy Transformations
on:
  push:
    branches: [main]
```

```yaml
env:
  UTLXE_ADMIN_URL: ${{ secrets.UTLXE_ADMIN_URL }}
  UTLXE_ADMIN_KEY: ${{ secrets.UTLXE_ADMIN_KEY }}
  UTLXE_DATA_URL: ${{ secrets.UTLXE_DATA_URL }}

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Build bundle
        run: |
          cd transformations
          zip -r ../bundle.zip schemas/ transformations/

      - name: Validate (dry run)
        run: |
          STATUS=$(curl -s -o /dev/null -w "%{http_code}" \
            -X POST \
            -H "X-Admin-Key: $UTLXE_ADMIN_KEY" \
            -F "file=@bundle.zip" \
            "$UTLXE_ADMIN_URL/admin/bundle/validate")
          if [ "$STATUS" != "200" ]; then
            echo "Validation failed with status $STATUS"
            exit 1
          fi

      - name: Deploy
        run: |
          curl -sf \
            -X POST \
            -H "X-Admin-Key: $UTLXE_ADMIN_KEY" \
            -F "file=@bundle.zip" \
            "$UTLXE_ADMIN_URL/admin/bundle"

      - name: Verify
        run: |
          for dir in test-data/*/; do
            NAME=$(basename "$dir")
            INPUT="$dir/input.json"
            EXPECTED="$dir/expected-output.xml"
            if [ -f "$INPUT" ]; then
              RESULT=$(curl -sf \
                -X POST \
                -H "X-Admin-Key: $UTLXE_ADMIN_KEY" \
                -H "Content-Type: application/json" \
                -d @"$INPUT" \
                "$UTLXE_ADMIN_URL/admin/transformations/$NAME/test")
              echo "$NAME: $RESULT"
            fi
          done
```

The workflow validates before deploying, and verifies after. If validation fails, the deployment never happens. If verification fails, the pipeline reports the error — the transformations are already deployed, but the team is notified.

9.5 Azure DevOps Pipeline

The structure is similar, using Azure CLI tasks for authentication:

```
trigger:
  branches:
    include:
      - main

pool:
  vmImage: 'ubuntu-latest'

variables:
  - group: utlxe-secrets

steps:
  - script: |
      cd transformations
      zip -r $(Build.ArtifactStagingDirectory)/bundle.zip schemas/ transformations/
    displayName: 'Build bundle'

  - script: |
      curl -sf -X POST \
        -H "X-Admin-Key: $(UTLXE_ADMIN_KEY)" \
        -F "file=@$(Build.ArtifactStagingDirectory)/bundle.zip" \
        "$(UTLXE_ADMIN_URL)/admin/bundle/validate"
    displayName: 'Validate'

  - script: |
      curl -sf -X POST \
        -H "X-Admin-Key: $(UTLXE_ADMIN_KEY)" \
        -F "file=@$(Build.ArtifactStagingDirectory)/bundle.zip" \
        "$(UTLXE_ADMIN_URL)/admin/bundle"
    displayName: 'Deploy'
```

Store `UTLXE_ADMIN_KEY` and `UTLXE_ADMIN_URL` in an Azure DevOps variable group marked as secret.

9.6 Rollback

Rollback is straightforward: re-deploy a previous bundle. Keep previous bundles as pipeline artifacts or in a git tag.

```
# Download previous bundle from artifact storage
# Re-deploy it
curl -X POST -H "X-Admin-Key: $KEY" \
  -F "file=@bundle-v1.2.3.zip" \
  http://<admin>:8081/admin/bundle
```

The bundle upload is atomic — the old transformations are replaced in one operation. There is no partial state.

Alternatively, use the export endpoint to create a backup before deploying:

```
# Backup current state
curl -H "X-Admin-Key: $KEY" \
  http://<admin>:8081/admin/bundle -o backup-before-deploy.zip

# Deploy new version
curl -X POST -H "X-Admin-Key: $KEY" \
  -F "file=@new-bundle.zip" \
  http://<admin>:8081/admin/bundle

# If something goes wrong, rollback
curl -X POST -H "X-Admin-Key: $KEY" \
  -F "file=@backup-before-deploy.zip" \
  http://<admin>:8081/admin/bundle
```

9.7 Multi-Environment Promotion

Use the same bundle across environments with environment-specific configuration:

Environment	Validation policy	How
Development	`off`	Runtime override via Admin API
Staging	`warn`	`transform.yaml` config
Production	`strict`	`transform.yaml` config

The `.utlx` source and schemas are identical across environments. Only the `transform.yaml` varies (or the runtime override via the Admin API).

9.8 The Full Cycle

A commit to `main` triggers the pipeline. Within two minutes:

1. The bundle is built from the repository.
2. It is validated against the running UTLXe (dry run — does it compile?).
3. It is deployed (atomic replacement of all transformations).
4. Each transformation is tested with sample input.
5. The pipeline reports success or failure.

From commit to verified production deployment in under two minutes, with no custom Docker images, no container restarts, and no downtime.

10 Persistence and Scaling

This chapter explains how transformations survive container restarts and how to scale UTLXe for production throughput.

10.1 The Persistence Problem

Docker containers have an ephemeral filesystem. When a container restarts — due to a crash, a scale-to-zero event, or a redeployment — everything written inside the container is lost. This means the `/utlxe/data/` directory, where uploaded transformations and schemas are stored, is wiped clean.

This is a fundamental container property, not something UTLXe can fix internally. The solution is external storage.

10.2 Azure Files Volume Mount

The recommended approach for the Azure Marketplace is an Azure File Share mounted at `/utlxe/data/`. The mount is configured by the platform at the infrastructure level — before the container starts. UTLXe uses standard Java file I/O to read and write the directory. It has no knowledge that the directory is backed by a network file share.

```
Container filesystem (ephemeral):
  /utlxe/utlxe.jar             from Docker image, rebuilt on restart

Mount point (persistent):
  /utlxe/data/                  Azure File Share — survives restarts
    schemas/
      order.xsd
    transformations/
      invoice-to-ubl/
        invoice-to-ubl.utlx
      order-enrichment/
        order-enrichment.utlx
```

On restart, UTLXe scans `/utlxe/data/`, finds the transformations from the previous session, compiles them, and becomes ready. Zero manual intervention.

To enable persistent storage, check the "Enable persistent transformation storage" option during Marketplace deployment. This creates an Azure Storage Account and File Share, and configures the volume mount in the Container App.

10.3 CI/CD Re-Deploy Pattern

An alternative to persistent storage: let the CI/CD pipeline re-deploy transformations after every container start.

1. Container starts with an empty `/utlxe/data/`.
2. Health endpoint returns `ready: false`.
3. The CI/CD pipeline detects the not-ready state and uploads the bundle.
4. UTLXe compiles the transformations.
5. Health endpoint switches to `ready: true`.
6. Kubernetes routes traffic.

The bundle lives in the CI/CD system (a git repository or artifact store). The container is truly stateless — the pipeline is the source of truth.

10.4 Startup Sequence

The startup sequence depends on what is found on disk:

10.4.1 Open Mode (no `.utlar` file)

1. Start the HTTP server on port 8081 (health, metrics, admin API).
2. Scan `/utlxe/data/` for existing transformations and schemas (directory structure).
   - If found (volume mount from previous session): compile and register them. Load `transform.yaml` with messaging config.
   - If empty: wait for API uploads.
3. Reconcile Dapr components (if `--dapr-components-dir` is set): sync messaging config to Dapr.
4. Start the data plane on port 8085.
5. The readiness probe checks `ready == true` before Kubernetes routes traffic.
6. Admin API: **full access** — upload, delete, configure, sync.

10.4.2 Locked Mode (`bundle.utlar` found)

1. Start the HTTP server on port 8081 (health, metrics, admin API).
2. Detect `bundle.utlar` in `/utlxe/data/` — enter **locked mode**.
3. Read manifest from `.utlar`: version, checksum, created timestamp.
4. Unpack `.utlar` (ZIP): load all transformations, schemas, and `transform.yaml` configs.
5. Compile all transformations. Create validators from schema references.
6. Reconcile Dapr components from messaging config in the bundle.
7. Start the data plane on port 8085.
8. Health: `ready: true`, `mode: locked`.
9. Admin API: **read-only** — mutating endpoints return 403 `BUNDLE_LOCKED`. Operational endpoints (pause, resume, validation override, log management) remain available.

The mode is determined automatically by the presence of `bundle.utlar` on disk. No CLI flag needed — if CI/CD placed a `.utlar`, it is production.

The admin API is available from step 1 in both modes — health probes work before transformations are loaded.

10.5 Memory Sizing

UTLXe runs on the JVM with a configured heap size. The heap must be large enough to hold the message being processed plus the intermediate objects created during transformation.

Plan	Container	Heap	Max message
Starter	2 GB	1536 MB	50 KB
Professional	4 GB	3072 MB	200 KB

The heap is set to 75% of the container memory. The remaining 25% is for the JVM itself (metaspace, thread stacks, native memory) and the operating system.

The `-XX:+AlwaysPreTouch` JVM flag allocates the entire heap at startup. If the container does not have enough RAM, the process fails immediately with a clear error — rather than crashing hours later under load.

10.6 Horizontal Scaling

For higher throughput, deploy multiple container replicas behind the Azure load balancer. Each replica runs an independent UTLXe instance.

If using persistent storage, all replicas share the same Azure File Share. Upload transformations once — all replicas see the same files.

If using CI/CD re-deploy, the pipeline must deploy to each replica (or use a shared storage backend for the bundle).

Azure Service Bus distributes messages across consumers automatically. Event Hub uses consumer groups to distribute partitions across replicas.

10.7 Never Use Swap

Swap and garbage collection do not mix. When the JVM's garbage collector scans the heap, it touches every page. If those pages have been swapped to disk, each page access triggers a page fault — a disk I/O operation that takes milliseconds instead of nanoseconds. A GC pause that normally takes 20 milliseconds can take 20 *seconds* when pages are swapped.

The rule: **never run UTLXe with swap enabled.**

In Azure Container Apps, set the memory request equal to the memory limit:

```
resources:
  requests:
    memory: "2Gi"
  limits:
    memory: "2Gi"
```

This ensures the container gets exactly the memory it requests, with no swap. If the container exceeds its memory limit, Kubernetes kills it (OOM) — which is faster to recover from than swap thrashing.

10.8 Poison Messages

A poison message is an input that is too large or too malformed to process. Without protection, a single 2 GB message can exhaust the JVM heap and crash the container.

The `maxInputSize` configuration rejects oversized messages before parsing:

```
maxInputSize: 5MB
```

Messages larger than this limit are rejected immediately with a 413 status code. The Dapr sidecar receives the error, and Service Bus moves the message to the dead-letter queue after max retries.

The default is 5 MB. Increase it only if you know your messages are legitimately larger, and verify that the heap has enough room.

11 Security

This chapter covers the security model of a UTLXe deployment: network isolation, authentication, secrets management, and the principle of least privilege.

11.1 Network Architecture

UTLXe uses port separation to isolate management traffic from data traffic:

Port	Scope	Contains
8085	Public (via ingress)	Data plane — message processing only
8081	Internal (VNet only)	Admin API, health probes, Prometheus metrics

The Container App ingress exposes only port 8085 to the outside world. Port 8081 is accessible only within the VNet, which means:

- Client applications can send messages for transformation but cannot upload, delete, or modify transformations.
- Operators and CI/CD pipelines access the admin API from within the VNet (or via VPN/private endpoint).
- Prometheus scrapes metrics from port 8081 inside the VNet.
- Kubernetes probes check health on port 8081 inside the VNet.

11.2 Admin API Authentication

The admin API is protected by an API key. Every request to `/admin/*` must include:

```
X-Admin-Key: <value of UTLXE_ADMIN_KEY>
```

If `UTLXE_ADMIN_KEY` is not set, all admin endpoints return 403. The API is locked by default — there is no open-by-default behavior.

Store the key as a Container App secret:

```
az containerapp secret set \
  -n utlxe -g myResourceGroup \
  --secrets admin-key=<your-secret-key>
```

Reference it in the environment:

```
az containerapp update \
  -n utlxe -g myResourceGroup \
  --set-env-vars UTLXE_ADMIN_KEY=secretref:admin-key
```

Container App secrets are encrypted at rest and injected as environment variables at runtime. They are not visible in logs or in the container filesystem.

To rotate the key, update the secret and restart the container. No code change is needed.

11.3 Non-Root Container

The UTLXe Docker image runs as a non-root user (`utlxe:utlxe`). The Dockerfile creates a dedicated user and group:

```
RUN groupadd -r utlxe && useradd -r -g utlxe utlxe
USER utlxe
```

The container has no elevated privileges. The only writable directory is `/utlxe/data/`, where the Admin API stores uploaded transformations and schemas.

11.4 Secrets in Transformations

Transformations sometimes need secrets — API keys, connection strings, tokens. Never hardcode secrets in `.utlx` files. Instead, use the `env()` function to read environment variables:

```
%utlx 1.0
input json
output json
---
{
  ...$input,
  authHeader: concat("Bearer ", env("API_TOKEN"))
}
```

The `API_TOKEN` value comes from a Container App secret, injected as an environment variable. The `.utlx` source can be committed to version control without exposing the secret.

The `env()` function is restricted in the CLI for security, but available in the engine (UTLXe) where environment variables are controlled by the deployment configuration.

11.5 VNet Integration

For production deployments, place the Container App Environment in a VNet:

- The admin port (8081) is accessible only from within the VNet.
- The data plane ingress (8085) can be configured as internal (VNet only) or external (public with TLS).
- Service Bus and Storage Account connections use private endpoints — no traffic over the public internet.

A typical production network layout:

UTLXe Container App	VNet, private subnet
Admin access	VNet only (or via Azure Bastion / VPN)
Data plane ingress	Internal or external, TLS terminated by Azure
Service Bus	Private endpoint in same VNet
Storage Account	Private endpoint in same VNet (for Azure Files)
Prometheus	In same VNet, scrapes port 8081

11.6 TLS and HTTPS

UTLXe itself runs HTTP on both ports. TLS termination is handled by the Container App ingress for port 8085. Internal traffic between Dapr, UTLXe, and other services within the VNet does not need TLS — it is already isolated by the network boundary.

If your security policy requires TLS on internal traffic, configure the Container App Environment with mTLS (mutual TLS), which encrypts all traffic between containers in the environment.

12 Operations

This chapter covers day-to-day operational tasks: updating transformations without downtime, handling incidents, and managing validation at runtime.

12.1 Updating a Transformation

Upload a new version of the same transformation name. The update is atomic and zero-downtime:

```
curl -X POST \
  -H "X-Admin-Key: $KEY" \
  -F "source=@invoice-to-ubl-v2.utlx" \
  http://<admin>:8081/admin/transformations/invoice-to-ubl
```

What happens internally:

1. UTLXe compiles the new source. If compilation fails, the upload is rejected and the running version is untouched.
2. The new compiled version atomically replaces the old one in the registry.
3. Messages that are currently being processed on the old version complete normally — they hold a reference to the old compiled code.
4. New messages arriving after the swap use the new version.

There is no restart, no mode switch, and no window where messages are dropped.

12.2 Pause and Resume

During an incident, you may need to stop processing for a specific transformation without removing it. Pausing preserves the compiled transformation and its on-disk state while stopping traffic.

Pause:

```
curl -X POST \
  -H "X-Admin-Key: $KEY" \
  http://<admin>:8081/admin/transformations/invoice-to-ubl/pause
```

While paused:

- Binding input: the data plane returns 503 for that transformation. Other transformations continue normally.
- Pub/sub input: the data plane returns 429 with a `Retry-After` header. Combined with a Dapr Resiliency circuit breaker, this prevents dead-lettering during maintenance windows.
- Messages stay in Service Bus until the transformation is resumed — they are not consumed or dead-lettered.
- The transformation stays compiled in memory and on disk — resume is instant.
- The listing shows `"status": "paused"`.

For global maintenance (pause everything), UTLXe can return 503 from `/healthz`, which causes Dapr to stop all bindings and subscriptions. See the Dapr Resiliency section in the architecture documentation.

Resume:

```
curl -X POST \
  -H "X-Admin-Key: $KEY" \
  http://<admin>:8081/admin/transformations/invoice-to-ubl/resume
```

Processing resumes immediately. Queued messages on Service Bus are delivered again.

12.3 Incident Response Workflow

A complete incident lifecycle:

1. **Detect** — Grafana alert fires on elevated error rate.
2. **Diagnose** — Check the error ring buffer:

```
curl -H "X-Admin-Key: $KEY" \
  .../admin/transformations/invoice-to-ubl/errors
```

3. **Contain** — Pause the transformation:

```
curl -X POST -H "X-Admin-Key: $KEY" \
  .../admin/transformations/invoice-to-ubl/pause
```

4. **Fix** — Edit the `.utlx` source to handle the failing case.
5. **Deploy** — Upload the fix:

```
curl -X POST -H "X-Admin-Key: $KEY" \
  -F "source=@invoice-to-ubl-fixed.utlx" \
  .../admin/transformations/invoice-to-ubl
```

6. **Verify** — Test with the input that caused the failure:

```
curl -X POST -H "X-Admin-Key: $KEY" \
  -d '{"orderId":"12345","customer":null}' \
  .../admin/transformations/invoice-to-ubl/test
```

7. **Resume** — Bring the transformation back online:

```
curl -X POST -H "X-Admin-Key: $KEY" \
  .../admin/transformations/invoice-to-ubl/resume
```

The entire cycle — from detection to resume — can happen in minutes without a container restart.

12.4 Validation Override

Schema validation catches malformed messages before they reach the transformation logic. But during an incident, validation might be too strict — blocking messages that are technically valid but don't match the expected schema exactly.

Set a runtime override:

```
curl -X POST \
  -H "X-Admin-Key: $KEY" \
  -H "Content-Type: application/json" \
  -d '{"policy": "off"}' \
  http://<admin>:8081/admin/transformations/invoice-to-ubl/validation
```

Check the effective state:

```
curl -H "X-Admin-Key: $KEY" \
  http://<admin>:8081/admin/transformations/invoice-to-ubl/validation
```

```
{
  "effective_policy": "off",
  "source": "runtime-override",
  "config_policy": "strict",
  "header_policy": "strict"
}
```

Remove the override to revert to the configured policy:

```
curl -X DELETE \
  -H "X-Admin-Key: $KEY" \
  http://<admin>:8081/admin/transformations/invoice-to-ubl/validation
```

The override is ephemeral — it is not written to disk and disappears on container restart. The precedence chain is:

Level	Set by	Persists
Runtime override	Ops via Admin API	No — ephemeral
`transform.yaml` config	Ops at deploy time	Yes — on disk
`.utlx` header	Developer at dev time	Yes — in source
Default	—	`strict`

The highest-priority level that is set wins.

12.5 Updating Schemas

Replace a schema without recompiling transformations:

```
curl -X POST \
  -H "X-Admin-Key: $KEY" \
  -F "file=@order-v2.xsd" \
  http://<admin>:8081/admin/schemas/order.xsd
```

The new schema takes effect on the next message. Transformations that reference `order.xsd` do not need to be re-uploaded — schema resolution happens at validation time, not compile time.

12.6 Bulk Operations

Replace everything at once (atomic):

```
curl -X POST -H "X-Admin-Key: $KEY" \
  -F "file=@new-bundle.zip" \
  http://<admin>:8081/admin/bundle
```

Remove everything and start fresh:

```
curl -X DELETE -H "X-Admin-Key: $KEY" \
  http://<admin>:8081/admin/bundle
```

Export the current state as a backup:

```
curl -H "X-Admin-Key: $KEY" \
  http://<admin>:8081/admin/bundle -o backup.zip
```

12.7 Engine Configuration

View the current engine configuration:

```
curl -H "X-Admin-Key: $KEY" \
  http://<admin>:8081/admin/config
```

Update a runtime-safe field:

```
curl -X POST \
  -H "X-Admin-Key: $KEY" \
  -H "Content-Type: application/json" \
  -d '{"maxInputSize": "10MB"}' \
  http://<admin>:8081/admin/config
```

Fields that require a restart (like port numbers) are accepted but flagged in the response:

```
{"updated": ["maxInputSize"], "restart_required": []}
```

12.8 Production Locked Mode

When CI/CD deploys a `bundle.utlar` file to the data volume, UTLXe enters **locked mode**. The Admin API becomes read-only for all mutating endpoints — transformations, schemas, messaging config, and bundle uploads all return 403 with `BUNDLE_LOCKED`.

Operational endpoints continue to work: pause/resume, validation override, test, log management, and all GET endpoints.

This matches enterprise expectations: production deployments are immutable. Changes go through CI/CD, not the Admin API.

Detection is automatic — if `/utlxe/data/bundle.utlar` exists, UTLXe is locked. No CLI flag needed.

```
# Check the mode
curl -H "X-Admin-Key: $KEY" http://<admin>:8081/admin/info
```

```
{
  "mode": "locked",
  "bundle_version": "v3.2.1",
  "bundle_checksum": "sha256:a1b2c3..."
}
```

12.9 Runtime Log Management

During an incident, switch to DEBUG logging without restarting the container:

```
# Switch to DEBUG with auto-revert after 30 minutes
curl -X POST -H "X-Admin-Key: $KEY" \
  -H "Content-Type: application/json" \
```

```
  -d '{"level": "DEBUG", "revert_after_minutes": 30}' \
  http://<admin>:8081/admin/log/level
```

View recent log entries directly via the Admin API — no need to open the Azure portal:

```
# Last 50 entries
curl -H "X-Admin-Key: $KEY" "http://<admin>:8081/admin/logs?limit=50"

# Only errors
curl -H "X-Admin-Key: $KEY" "http://<admin>:8081/admin/logs?level=ERROR"

# Search for a specific MessageId
curl -H "X-Admin-Key: $KEY" "http://<admin>:8081/admin/logs?contains=msg-abc-123"
```

The log buffer holds the last 5000 entries in memory. Zero disk I/O. The auto-revert feature ensures DEBUG does not stay on accidentally — the level reverts to the previous value after the specified time.

12.10 Messaging Configuration

Configure Dapr queues, topics, and Event Hubs per transformation via the Admin API. Changes are staged as drafts and pushed to Dapr via explicit sync:

```
# 1. Set messaging (staged as draft --- Dapr not touched)
curl -X POST -H "X-Admin-Key: $KEY" \
  -H "Content-Type: application/json" \
  -d '{"input": {"queue": "orders-in"}, "output": {"topic": "processed"}}' \
  http://<admin>:8081/admin/transformations/orders-in/messaging

# 2. Test via HTTP (no Dapr needed --- the transformation works via HTTP)
curl -X POST -H "X-Admin-Key: $KEY" \
  -d '{"orderId": "123"}' \
  http://<admin>:8081/admin/transformations/orders-in/test

# 3. Sync to Dapr (creates binding YAML, messages start flowing)
curl -X POST -H "X-Admin-Key: $KEY" \
  http://<admin>:8081/admin/transformations/orders-in/sync
```

The stage-then-sync model lets you test transformations via HTTP before connecting them to real queues. Sync is the “go live” switch.

13 Monitoring and Observability

Production deployments need visibility into what UTLXe is doing. This chapter covers health probes, Prometheus metrics, and how to set up monitoring dashboards.

13.1 Health Endpoint

The health endpoint runs on port 8081 and serves two purposes: Kubernetes probes and operational status checks.

```
curl http://<internal-ip>:8081/health
```

```
{
  "status": "UP",
  "transformations": 3,
  "ready": true
}
```

The fields:

`status`	`"UP"` if the process is alive. Used by the liveness probe.
`transformations`	Number of compiled transformations in the registry.
`ready`	`true` when at least one transformation is loaded and compiled. Used by the readiness probe.

13.1.1 Liveness vs. Readiness

Probe	Checks	Action if fails
Liveness	`status == "UP"`	Kubernetes restarts the container
Readiness	`ready == true`	Kubernetes stops routing traffic to this instance

A container that just started but has not received its bundle yet is **alive but not ready**. Traffic is held until transformations are uploaded and compiled. This prevents errors during the deployment window.

13.2 Prometheus Metrics

UTLXe exposes Prometheus metrics on the same port:

```
curl http://<internal-ip>:8081/metrics
```

13.2.1 Per-Transformation Metrics

```
utlxe_messages_total{transform="invoice-to-ubl"} 12345
utlxe_transform_duration_seconds{transform="invoice-to-ubl",quantile="0.50"} 0.002
utlxe_transform_duration_seconds{transform="invoice-to-ubl",quantile="0.95"} 0.005
utlxe_transform_duration_seconds{transform="invoice-to-ubl",quantile="0.99"} 0.012
utlxe_errors_total{transform="invoice-to-ubl"} 3
```

13.2.2 Engine-Level Metrics

```
utlxe_active_threads 4
utlxe_heap_used_bytes 524288000
utlxe_uptime_seconds 86400
```

These metrics are standard Prometheus format and can be scraped by any Prometheus-compatible system.

13.3 Three Monitoring Options

UTLXe's Prometheus endpoint (`GET /metrics`) works with any Prometheus-compatible system. On Azure, there are three practical options:

Option	Effort	Best for
Azure Monitor + Managed Grafana	Low — fully managed	Azure-native teams, production
Self-hosted Prometheus + Grafana	Medium — you manage the stack	Multi-cloud, existing Prometheus
Admin API only	Zero — built into UTLXe	Quick checks, small deployments

13.4 Option 1: Azure Monitor Managed Prometheus + Managed Grafana

This is the recommended approach for Azure deployments. Fully managed, no infrastructure to maintain.

13.4.1 Step 1: Create Azure Monitor Workspace + Managed Grafana

```
RESOURCE_GROUP="rg-utlxe-monitoring"
LOCATION="westeurope"

# Create Azure Monitor workspace (stores Prometheus metrics)
az monitor account create \
  --name utlxe-prometheus \
  --resource-group $RESOURCE_GROUP \
  --location $LOCATION

# Create Managed Grafana instance
az grafana create \
  --name utlxe-grafana \
  --resource-group $RESOURCE_GROUP \
  --location $LOCATION

# Link Grafana to the Azure Monitor workspace
az grafana data-source create \
  --name utlxe-grafana \
  --resource-group $RESOURCE_GROUP \
  --definition '{
    "name": "Azure Managed Prometheus",
    "type": "prometheus",
    "url": "https://utlxe-prometheus-XXXX.westeurope.prometheus.monitor.azure.com"
  }'
```

Portal alternative: Search "Azure Monitor workspace" > Create. Then search "Azure Managed Grafana" > Create. Link them under Grafana > Settings > Data sources.

13.4.2 Step 2: Enable Prometheus Scraping on Container Apps

Azure Container Apps can scrape Prometheus endpoints automatically when Azure Monitor is configured:

```
# Get the Container App Environment resource ID
ENV_ID=$(az containerapp env show \
  --name utlxe-env --resource-group rg-utlxe-prd \
  --query id -o tsv)

# Enable metrics collection
```

```
az monitor account update \
  --name utlxe-prometheus \
  --resource-group $RESOURCE_GROUP \
  --linked-environment "$ENV_ID"
```

Azure Monitor scrapes `GET /metrics` on port 8081 every 30 seconds. No sidecar or annotation needed — Container Apps has built-in Prometheus scraping support.

13.4.3 Step 3: Import the UTLXe Grafana Dashboard

Open Managed Grafana (URL from `az grafana show`) and import the following dashboard.

13.4.3.1 Dashboard Layout

Row 1 — Message Throughput (the business view):

Panel	PromQL	Description
Messages/sec	`rate(utlxe_messages_total[5m])`	Messages processed per second, per transformation
Errors/sec	`rate(utlxe_errors_total[5m])`	Errors per second
Error %	`rate(utlxe_errors_total[5m]) / rate(utlxe_messages_total[5m]) * 100`	Error rate as percentage
Total messages (24h)	`increase(utlxe_messages_total[24h])`	Volume over last 24 hours

Row 2 — Latency (the performance view):

Panel	PromQL	Description
p50 latency	`utlxe_transform_duration_seconds{quantile="0.50"}`	Median — what most messages experience
p95 latency	`utlxe_transform_duration_seconds{quantile="0.95"}`	Tail — the slow 5%
p99 latency	`utlxe_transform_duration_seconds{quantile="0.99"}`	Worst case — may trigger investigation

Row 3 — Engine Health:

Panel	PromQL / Source	Description
Transformations loaded	`utlxe_transformations_loaded`	Should be > 0 always
Heap usage	`utlxe_heap_used_bytes / utlxe_heap_max_bytes * 100`	JVM heap percentage

Uptime	`utlxe_uptime_seconds`	Time since last restart — drops indicate restarts
Container CPU	Azure Monitor (Container Insights)	CPU usage from the platform
Container Memory	Azure Monitor (Container Insights)	Memory from the platform

Row 4 — Per-Transformation Breakdown:

A table panel showing each transformation with:
- Name, status (ready/paused)
- Messages processed, errors, error rate
- Average latency

PromQL: `utlxe_messages_total` grouped by `transform` label.

13.4.4 Step 4: Configure Alert Rules

```
# Error rate alert
az monitor metrics alert create \
  --name utlxe-error-rate \
  --resource-group $RESOURCE_GROUP \
  --condition "avg rate(utlxe_errors_total[5m]) > 0.01" \
  --description "UTLXe error rate exceeds 1%" \
  --severity 2 \
  --action-group oncall-team

# Zero transformations alert (restart without bundle)
az monitor metrics alert create \
  --name utlxe-no-transforms \
  --resource-group $RESOURCE_GROUP \
  --condition "utlxe_transformations_loaded == 0" \
  --description "UTLXe has no transformations loaded" \
  --severity 1 \
  --action-group oncall-team
```

Recommended alert rules:

Alert	Condition	Severity	Action
Error rate	> 1% for 5 min	Warning	Notify on-call — check error ring buffer
p99 latency	> 500ms for 5 min	Warning	Investigate — GC pressure or large messages
Heap usage	> 80%	Warning	Consider scaling up or increasing heap
Zero transforms	**13.5 0 for 2 min**	Critical	Bundle not loaded — check persistence/CI/CD
Container restart	uptime drops	Info	Expected during deployment, investigate if unexpected

13.6 Option 2: Self-Hosted Prometheus + Grafana

If you already have a Prometheus stack or need multi-cloud monitoring:

13.6.1 Deploy Prometheus in the Same VNet

```yaml
# prometheus.yml (scrape config)
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: 'utlxe'
    static_configs:
      - targets:
        - 'utlxe-prd.internal.<env-id>.westeurope.azurecontainerapps.io:8081'
        - 'utlxe-acc.internal.<env-id>.westeurope.azurecontainerapps.io:8081'
    metrics_path: '/metrics'
```

Deploy Prometheus as a Container App in the same environment, or use an external Prometheus that can reach the VNet.

13.6.2 Deploy Grafana

```
# Deploy Grafana as a Container App
az containerapp create \
  --name grafana \
  --resource-group rg-utlxe-monitoring \
  --environment utlxe-env \
  --image grafana/grafana:latest \
  --target-port 3000 \
  --ingress external \
  --env-vars GF_SECURITY_ADMIN_PASSWORD=secretref:grafana-password
```

Then add the Prometheus data source in Grafana and import the same dashboard panels described above.

13.7 Option 3: Admin API Only (No External Tools)

For small deployments or quick checks, the Admin API provides everything without external infrastructure:

```
# Health check
curl http://<admin>:8081/health

# Transformation metrics
curl -H "X-Admin-Key: $KEY" http://<admin>:8081/admin/transformations
# → messages_processed, errors per transformation

# Recent errors
curl -H "X-Admin-Key: $KEY" \
  "http://<admin>:8081/admin/transformations/orders-in/errors?limit=10"

# Log entries
curl -H "X-Admin-Key: $KEY" \
  "http://<admin>:8081/admin/logs?level=ERROR&limit=20"

# Engine info (mode, uptime, bundle version)
curl -H "X-Admin-Key: $KEY" http://<admin>:8081/admin/info
```

This is sufficient for a single UTLXe instance in development. For production with multiple instances, use Option 1 or 2 for aggregate views and alerting.

13.8 Monitoring Multiple UTLXe Instances

When running multiple UTLXe instances (one per flow), each exposes its own `/metrics` endpoint. Prometheus scrapes all of them. Grafana shows a fleet view:

- **Dashboard variable:** add a `$instance` variable that selects the UTLXe instance from the `job` or `instance` label.
- **Aggregate panels:** `sum(rate(utlxe_messages_total[5m]))` shows total throughput across all instances.
- **Per-instance panels:** `rate(utlxe_messages_total{instance=~"$instance"}[5m])` filters to a single instance.

For centralized management of multiple instances, see the UTLXc control plane (Chapter 11 in the feature roadmap, EF11).

13.9 Error Diagnosis

Prometheus gives you counters, but not the actual error messages. For quick diagnosis, use the error ring buffer:

```
curl -H "X-Admin-Key: $KEY" \
  http://<admin>:8081/admin/transformations/invoice-to-ubl/errors?limit=10
```

```
{
  "errors": [
    {
      "timestamp": "2026-05-05T14:32:01Z",
      "message": "Null reference: $input.customer.address",
      "line": 14,
      "input_preview": "{\"orderId\":\"12345\",\"customer\":{\"name\":\"Acme\"}}"
    }
  ],
  "total_errors": 47,
  "showing": 10
}
```

The `input_preview` is truncated to 200 characters to avoid exposing full message payloads. This is usually enough to identify the pattern that causes failures.

The typical diagnosis workflow:

1. Notice elevated error rate in Grafana.
2. Check the error ring buffer for the affected transformation.
3. Identify the pattern (missing field, unexpected type, schema mismatch).
4. Pause the transformation if needed (Chapter 6).
5. Fix and re-upload.
6. Test with sample input.
7. Resume.

13.10 Runtime Log Access

For deeper investigation, access log entries directly via the Admin API without opening the Azure portal:

```
# Last 50 log entries
curl -H "X-Admin-Key: $KEY" \
  "http://<admin>:8081/admin/logs?limit=50"

# Only errors
```

```
curl -H "X-Admin-Key: $KEY" \
  "http://<admin>:8081/admin/logs?level=ERROR"

# Search for a specific message
curl -H "X-Admin-Key: $KEY" \
  "http://<admin>:8081/admin/logs?contains=orders-in"
```

For Dapr traffic tracing, switch to DEBUG level at runtime:

```
# Enable DEBUG with auto-revert after 30 minutes
curl -X POST -H "X-Admin-Key: $KEY" \
  -H "Content-Type: application/json" \
  -d '{"level": "DEBUG", "revert_after_minutes": 30}' \
  http://<admin>:8081/admin/log/level
```

At DEBUG level, UTLXe logs the full request/response flow for every Dapr message: input headers, CloudEvents metadata, output URL, response timing, and Dapr response body on failure. This is the primary tool for diagnosing message routing issues.

The log buffer holds 5000 entries in memory. No disk I/O — performance impact is negligible. The auto-revert ensures DEBUG does not stay on accidentally.

13.11 Metrics Design: What UTLXe provides vs. what's external

UTLXe provides raw counters (messages processed, errors) per transformation. External tools handle everything else:

UTLXe provides	**External (Prometheus / Azure Monitor)**
Total message count per transformation	Message rate (msg/sec) via `rate()`
Total error count per transformation	Error rate and error percentage
Error details (last 100, ring buffer)	Latency percentiles (p50, p95, p99)
Prometheus-format counters on `/metrics`	Dashboards, graphs, alerting
Log buffer (last 5000, Admin API)	Long-term log storage (Azure Monitor)

Resettable counters were considered and not implemented. Prometheus computes rates from monotonic counters automatically — a `rate()` query over a time window is more meaningful than a counter that was "reset 3 hours ago."

14 Troubleshooting

This chapter lists the most common problems, their symptoms, and how to fix them.

14.1 Container Starts But `ready=false`

Symptom: Health endpoint returns `{"status":"UP", "transformations":0, "ready":false}`. No traffic is routed.

Cause: No transformations are loaded. The container started empty.

Fix:

- If using persistent storage: check that the Azure File Share is mounted correctly. Run `az containerapp show` and verify the volume mount configuration.
- If using CI/CD re-deploy: check that the pipeline ran and uploaded the bundle.
- Manual fix: upload a transformation via the Admin API.

14.2 Transformation Upload Fails (400)

Symptom: `POST /admin/transformations/{name}` returns 400 with an error message.

Cause: Syntax error in the `.utlx` source. The compilation failed.

Fix: Read the error response — it includes the line number and a description of the problem.

```
{
  "status": "rejected",
  "errors": [
    {"transformation": "invoice", "line": 14, "message": "Unknown function: concatX"}
  ]
}
```

Test locally before uploading: `echo '{}' | utlx -e 'your expression'` catches syntax errors immediately.

14.3 Messages Failing (500 on Data Plane)

Symptom: The data plane returns 500 for some or all messages. Error count increases in Prometheus.

Cause: Runtime error in the transformation — null reference, type mismatch, missing field.

Diagnose:

```
curl -H "X-Admin-Key: $KEY" \
  http://<admin>:8081/admin/transformations/invoice-to-ubl/errors
```

This shows the last 100 errors with the input that caused each failure.

Fix: Upload a corrected transformation. Test with the failing input before resuming:

```
curl -X POST -H "X-Admin-Key: $KEY" \
  -d '{"the":"failing input"}' \
  http://<admin>:8081/admin/transformations/invoice-to-ubl/test
```

14.4 High Latency

Symptom: p99 latency exceeds 100ms or response times are inconsistent.

Cause 1 — Large messages: Parsing and serialization dominate for messages above 50 KB.

Check average message size and consider whether the transformation can be simplified, or upgrade to the Professional plan for more memory.

Cause 2 — GC pressure: The heap is too small for the workload.

Check `utlxe_heap_used_bytes` in Prometheus. If it stays above 80% of the max, upgrade the plan.

Cause 3 — Swap: The container has swap enabled.

Fix: set memory request equal to memory limit in the Container App configuration. See Chapter 7 for details.

14.5 Out of Memory (OOM Kill)

Symptom: Container restarts unexpectedly. Container logs show `Killed` or the exit code is 137.

Cause: A message (or burst of messages) consumed more heap than available. The JVM exceeded the container memory limit, and Kubernetes killed the process.

Fix:

- Set `maxInputSize` to reject oversized messages before parsing.
- Upgrade to a larger plan.
- Check for transformations that create large intermediate objects (deeply nested `map` of `map` operations).

14.6 Admin API Returns 403

Symptom: All admin endpoints return 403 Forbidden.

Cause 1: `UTLXE_ADMIN_KEY` is not set. When no key is configured, all admin endpoints are locked.

Cause 2: The `X-Admin-Key` header value does not match the environment variable.

Fix: Verify the key:

```
# Check if the env var is set
az containerapp show -n utlxe -g myResourceGroup \
  --query "properties.template.containers[0].env"
```

14.7 Transformations Lost After Restart

Symptom: After a container restart, health shows `transformations: 0`.

Cause: No persistent storage is configured. The container filesystem is ephemeral.

Fix:

- Enable persistent storage (Azure Files volume mount) by redeploying with the "Enable persistent transformation storage" option.
- Or set up a CI/CD pipeline to re-deploy the bundle after each container start.

14.8 Dapr Not Delivering Messages

Symptom: Messages are in the Service Bus queue but UTLXe is not processing them.

Cause 1 — Not ready: UTLXe health shows `ready: false`. Dapr waits for the app to be ready before delivering messages.

Fix: Upload transformations. Dapr begins delivering once `ready: true`.

Cause 2 — Binding name mismatch: The Dapr component name does not match a transformation name, and no `X-UTLXe-Transform` header is configured.

Fix: Verify that the Dapr component `metadata.name` matches the transformation name, or configure the transform header override.

Cause 3 — Transformation is paused: UTLXe returns 503, Dapr does not acknowledge.

Fix: Resume the transformation:

```
curl -X POST -H "X-Admin-Key: $KEY" \
  http://<admin>:8081/admin/transformations/{name}/resume
```

14.9 Schema Validation Failing Unexpectedly

Symptom: Messages that used to work are now rejected by validation.

Cause: The schema was updated but the incoming messages still use the old format.

Quick fix: Disable validation temporarily:

```
curl -X POST -H "X-Admin-Key: $KEY" \
  -d '{"policy":"off"}' \
  http://<admin>:8081/admin/transformations/{name}/validation
```

Proper fix: Update the transformation to handle both the old and new message formats, or coordinate the schema change with the message producer.

14.10 Diagnostic Commands Reference

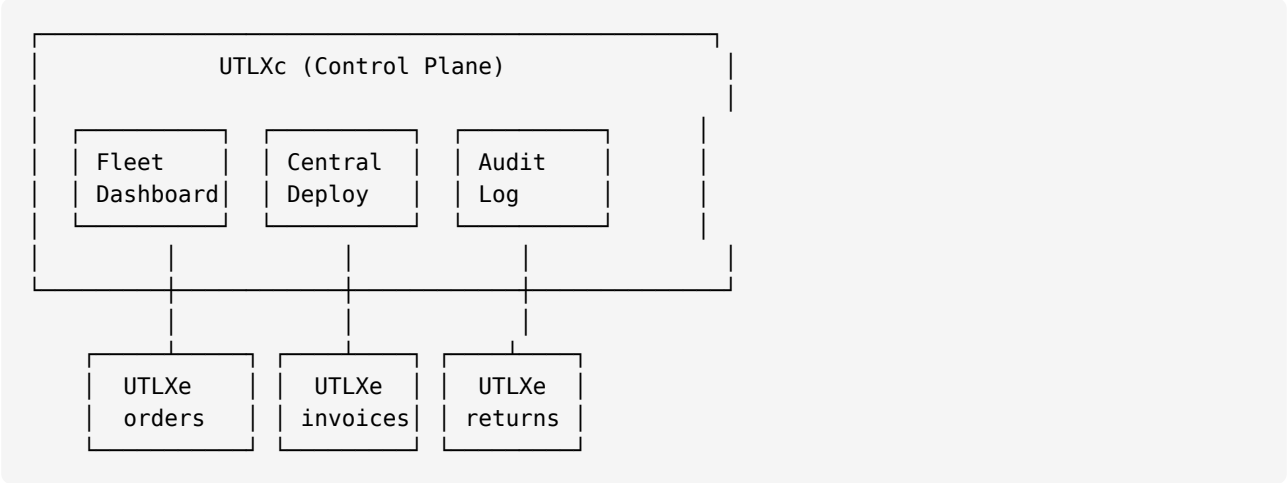
```
# Health and readiness
curl http://<internal>:8081/health

# Engine info (version, uptime, config)
curl -H "X-Admin-Key: $KEY" http://<internal>:8081/admin/info

# List transformations with status
curl -H "X-Admin-Key: $KEY" http://<internal>:8081/admin/transformations

# Recent errors for a transformation
curl -H "X-Admin-Key: $KEY" \
  http://<internal>:8081/admin/transformations/{name}/errors

# Test a transformation
curl -X POST -H "X-Admin-Key: $KEY" \
  -d '{"test":"data"}' \
  http://<internal>:8081/admin/transformations/{name}/test

# Effective validation state
curl -H "X-Admin-Key: $KEY" \
  http://<internal>:8081/admin/transformations/{name}/validation

# Container logs
az containerapp logs show -n utlxe -g myResourceGroup --follow
```

15 Roadmap: What's Next

UTLXe is the data plane — it transforms messages. This chapter describes planned capabilities that extend UTLXe into a complete enterprise integration platform.

15.1 UTLXc: The Control Plane

When you run a dozen UTLXe instances (one per flow or flow group), managing them individually becomes a burden. UTLXc is a separate component — the **control plane** — that provides centralized management for a fleet of UTLXe data planes.



15.1.1 What UTLXc Provides

Capability	Description
Fleet dashboard	All UTLXe instances at a glance — health, loaded transformations, throughput, errors.
Centralized deployment	Deploy a `.utlar` bundle to all instances tagged "invoices" with one command.
Aggregate monitoring	Error rates and throughput across all instances — not per-instance.
Coordinated rollout	Deploy to one instance, verify health for 5 minutes, then roll to the next.
Audit log	Who deployed what, when, to which instance.
RBAC via Azure AD	Viewer, deployer, and admin roles — integrated with Azure Active Directory.

15.1.2 How It Works with UTLXe

UTLXc does not replace UTLXe's Admin API — it calls it. Every UTLXe instance remains independently operational. If UTLXc goes down, each UTLXe continues processing messages. UTLXc is a management convenience, not a runtime dependency.

In the first version, UTLXc discovers instances from configuration and calls their Admin APIs. In a future version, UTLXe instances register with UTLXc on startup and send periodic heartbeats — enabling automatic discovery and real-time fleet status.

15.1.3 Marketplace Offering

Offering	Includes	Target
UTLXe (standalone)	One UTLXe instance	Small teams, 1–3 flows
UTLXe + UTLXc (fleet)	UTLXc control plane + N UTLXe instances	Enterprise, 5+ flows

UTLXc is the enterprise upsell. Start with standalone UTLXe, then upgrade when you outgrow manual deployment.

15.2 .NET SDK and BizTalk Integration

UTLXe speaks gRPC alongside HTTP. A .NET SDK will allow C# applications to call UTLXe transformations directly — without Dapr, without HTTP, without message queues.

Use cases:

- **BizTalk Server migration:** replace BizTalk maps with UTL-X transformations. The .NET SDK calls UTLXe from a BizTalk pipeline component.
- **Azure Functions:** call UTLXe from a C# Function via gRPC — synchronous request/response.
- **ASP.NET APIs:** transform request or response payloads inline.

The gRPC transport is already implemented (EF07). The .NET SDK wraps it with a typed C# client:

```
// Future: .NET SDK usage
var client = new UtlxeClient("utlxe.internal:9090");
var result = await client.TransformAsync("invoice-to-ubl", invoiceJson);
Console.WriteLine(result.Output);
```

15.3 Encoding Support (UTF-16, Shift-JIS)

UTLXe currently processes messages as UTF-8 strings. For BizTalk and SAP integrations, UTF-16 and other encodings are common. Planned encoding support will pass byte arrays through the transformation pipeline instead of strings — preserving the original encoding from input to output.

This is required before BizTalk and SAP CPI integrations can handle non-UTF-8 data correctly.

15.4 OpenTelemetry

UTLXe currently supports W3C Trace Context propagation (traceparent/tracestate headers forwarded through Dapr). Future work will add full OpenTelemetry span creation — each transformation execution becomes a span in a distributed trace, visible in Azure Application Insights or Jaeger.

15.5 What You Can Do Today

Everything described in this book — from deployment to monitoring to CI/CD — works today. The features above are planned extensions. Your investment in UTL-X transformations, Dapr configuration, and CI/CD pipelines carries forward — nothing changes when these capabilities are added.

16 Appendix A: API Reference

This appendix lists every endpoint on both ports with request format, response format, and status codes.

16.1 Authentication

All `/admin/*` endpoints require the header `X-Admin-Key: {value}`. The value must match the `UTLXE_ADMIN_KEY` environment variable. If the variable is not set, all admin endpoints return 403.

The `/health`, `/metrics`, and `/api/*` endpoints do not require authentication.

16.2 Admin Port (8081)

16.2.1 Bundle Endpoints

Method	Path	Description
`POST`	`/admin/bundle`	Upload a `.zip` bundle. Replaces all transformations and schemas atomically.
`GET`	`/admin/bundle`	Export the current state as a downloadable `.zip`.
`DELETE`	`/admin/bundle`	Remove all transformations and schemas. Returns to empty state.
`POST`	`/admin/bundle/validate`	Upload and validate without deploying (dry run).

16.2.2 Transformation Endpoints

Method	Path	Description
`GET`	`/admin/transformations`	List all deployed transformations with status and metrics.
`GET`	`/admin/transformations/{name}`	Get details: config, compile status, metrics, source.
`POST`	`/admin/transformations/{name}`	Deploy or update. Accepts `multipart/form-data` with `source` (required) and `config` (optional).
`POST`	`/admin/transformations/{name}/config`	Update config only. No recompile.
`DELETE`	`/admin/transformations/{name}`	Remove a single transformation.

16.2.3 Testing Endpoint

Method	Path	Description
`POST`	`/admin/transformations/{name}/test`	Execute with sample input. Returns output or error. Not counted in metrics.

16.2.4 Operational Endpoints

Method	Path	Description
`GET`	`/admin/info`	Engine version, uptime, config, persistence mode, transformation count.
`POST`	`/admin/transformations/{name}/pause`	Stop processing. Data plane returns 503. Transformation stays compiled.
`POST`	`/admin/transformations/{name}/resume`	Resume processing.
`GET`	`/admin/transformations/{name}/errors`	Recent errors (ring buffer, last 100). Accepts `?limit=N`.
`GET`	`/admin/config`	View current engine configuration.
`POST`	`/admin/config`	Update runtime-safe configuration fields.

16.2.5 Validation Override Endpoints

Method	Path	Description
GET	/admin/transformations/{name}/validation	Get effective validation state (policy, source, config default).
POST	/admin/transformations/{name}/validation	Set a runtime override. Body: {"policy":"off"}. Ephemeral — not persisted.
DELETE	/admin/transformations/{name}/validation	Remove override. Revert to config or header default.

16.2.6 Schema Endpoints

Method	Path	Description
GET	/admin/schemas	List all uploaded schemas with filename and size.
GET	/admin/schemas/{filename}	Download a schema file.
POST	/admin/schemas/{filename}	Upload or replace a schema. Accepts multipart/form-data with file.
DELETE	/admin/schemas/{filename}	Remove a schema.

16.2.7 Messaging Endpoints (EF10)

Configure Dapr messaging (queues, topics, Event Hubs) per transformation. Changes are staged as drafts and pushed to Dapr via sync.

Method	Path	Description
GET	/admin/dapr	Dapr sidecar status, integration mode (http-only/static/dynamic), loaded components.
GET	/admin/transformations/{name}/messaging	Messaging config with sync status and Dapr binding state.
POST	/admin/transformations/{name}/messaging	Set input/output messaging (queue/topic/eventhub). Saved as draft.
DELETE	/admin/transformations/{name}/messaging	Remove messaging config. Saved as draft.
POST	/admin/transformations/{name}/sync	Push this transformation's messaging to Dapr.
POST	/admin/sync	Push all draft transformations to Dapr.
GET	/admin/sync	Sync status overview for all transformations.

Messaging body uses the field name as discriminator — queue, topic, or eventhub:

```
{"input": {"queue": "orders-in"}, "output": {"topic": "processed-orders"}}
{"input": {"topic": "raw-invoices", "subscription": "utlxe"}, "output": {"queue":
"alerts"}}
{"input": {"eventhub": "telemetry", "consumerGroup": "utlxe"}, "output": {"queue":
"alerts"}}
```

16.2.8 Log Management Endpoints (EF12)

Runtime log level changes and in-memory log access. Allowed in locked mode.

Method	Path	Description
GET	/admin/log/level	Current root log level.

POST	/admin/log/level	Change log level. Body: {"level":"DEBUG","revert_after_minutes":30}.
GET	/admin/logs	Recent log entries from memory buffer. Params: ?limit=N&level=ERROR&contains=text&since=ISO8601.
DELETE	/admin/logs	Clear the log buffer.

16.2.9 Health Endpoints (no authentication)

Method	Path	Description
GET	/health	Returns status, transformation count, and ready flag.
GET	/health/ready	Readiness probe. Returns 200 when RUNNING with at least one transformation.
GET	/health/live	Liveness probe. Always returns 200.
GET	/metrics	Prometheus metrics in text exposition format.

16.3 Data Plane Port (8085)

Method	Path	Description
GET	/api/transformations	List available transformations (discovery). No authentication.
POST	/api/execute/{id}	Execute a transformation. Body: {"payload":"...","contentType":"application/json"}.
OPTIONS	/{bindingName}	Dapr binding probe. Always returns 200.
POST	/{bindingName}	Dapr input binding delivery. Routes to transformation by binding name.
GET	/dapr/subscribe	Dapr pub/sub subscription list. Derived from loaded transformations with topic config.
POST	/pubsub/{name}	Dapr pub/sub delivery. Accepts CloudEvents (structured or binary mode).

16.4 Production Locked Mode (EF09)

When bundle.utlar is found on the data volume, UTLXe enters locked mode. Mutating endpoints return 403:

Endpoint category	Open mode	Locked mode
Upload/delete transformations	Allowed	**403 BUNDLE_LOCKED**
Upload/delete bundle	Allowed	**403 BUNDLE_LOCKED**
Upload/delete schemas	Allowed	**403 BUNDLE_LOCKED**
Set/delete messaging	Allowed	**403 BUNDLE_LOCKED**
Pause/resume	Allowed	Allowed — operational
Validation override	Allowed	Allowed — operational
Test transformation	Allowed	Allowed — read-only
All GET endpoints	Allowed	Allowed — read-only
Log level / logs	Allowed	Allowed — diagnostic

16.5 Response Status Codes

Code	Meaning
200	Success.
400	Bad request — compilation error, invalid input, malformed ZIP.

403	Forbidden — missing or wrong `X-Admin-Key`, key not configured, or locked mode.
404	Transformation not found.
413	Message too large — exceeds `maxInputSize`.
429	Transformation paused — back off (pub/sub input). Includes `Retry-After` header.
500	Transformation runtime error (null reference, type mismatch, etc.).
503	Transformation not loaded or paused (binding input).

17 Appendix B: Configuration Reference

17.1 Environment Variables

Variable	Default	Description
`UTLXE_ADMIN_KEY`	*(none)*	Admin API authentication key. Required — if not set, all admin endpoints return 403.
`UTLXE_HEAP_SIZE`	`1536m`	JVM heap size. Starter: `1536m`. Professional: `3072m`.
`JAVA_OPTS`	*(see below)*	Additional JVM options. Default includes G1GC, container support, AlwaysPreTouch.

Default `JAVA_OPTS`:

```
-XX:+UseG1GC -XX:MaxGCPauseMillis=200 -XX:+UseContainerSupport -XX:+AlwaysPreTouch
```

17.2 Engine Configuration (engine.yaml)

The engine configuration file is optional. If present in the bundle, it overrides the defaults.

Field	Default	Description
`maxInputSize`	`5MB`	Maximum message size before rejection. Messages larger than this are rejected with 413.
`workers`	CPU cores	Worker thread pool size.
`healthPort`	`8081`	Health + admin API port.
`dataPort`	`8085`	Data plane port.
`grpcPort`	`9090`	gRPC port (when `--also-grpc` is used).

17.3 Port Map

Port	Container	Default	Purpose
8081	utlxe	configurable	Admin API + health + metrics (internal to VNet)
8085	utlxe	configurable	Data plane — Dapr bindings + direct HTTP
9090	utlxe	configurable	gRPC (optional, `--also-grpc`)
8088	utlxe-ui	configurable via `UI_PORT`	Admin Web UI (browser access)
3500	dapr	fixed	Dapr sidecar HTTP API (localhost only)

All UTLXe ports are configurable via CLI flags. The UI port is configurable via the `UI_PORT` environment variable. The Dapr sidecar port is fixed at 3500 by Dapr.

17.4 Transformation Configuration (transform.yaml)

Per-transformation configuration. Optional — if absent, defaults apply.

Field	Default	Description
`strategy`	`COMPILED`	Execution strategy: `TEMPLATE`, `COPY`, `COMPILED`, `AUTO`.
`validationPolicy`	`strict`	Input validation: `strict` (reject), `warn` (log), `off` (skip).
`maxConcurrent`	*(unlimited)*	Maximum concurrent executions for this transformation.
`outputBinding`	*(none)*	Dapr output binding name for sending transformed messages.

Inputs and schemas:

```
strategy: COMPILED
validationPolicy: strict
maxConcurrent: 4
inputs:
  - name: input
    schema: order.xsd
output:
  schema: invoice.xsd

# Messaging (EF10) — field name is the discriminator
input:
  queue: orders-in              # or topic: / eventhub:
  # subscription: utlxe         # required for topic input
  # consumerGroup: utlxe        # optional for eventhub (triggers pub/sub mode)
output_messaging:
  queue: orders-out             # or topic: / eventhub:
```

17.5 JVM Tuning Reference

Flag	Purpose
`-XX:+UseG1GC`	G1 garbage collector. Good for large heaps with pause-time targets.
`-XX:MaxGCPauseMillis=200`	Target maximum GC pause. G1 adjusts its behavior to meet this target.
`-XX:+UseContainerSupport`	Respect cgroup memory limits instead of reading host memory.
`-XX:+AlwaysPreTouch`	Allocate the entire heap at startup. Fail fast if not enough RAM.
`-Xmx${UTLXE_HEAP_SIZE}`	Maximum heap size. Set via `UTLXE_HEAP_SIZE` environment variable.

17.6 Container Plans

Plan	Container RAM	Heap	Max message	vCPU
Starter	2 GB	1536 MB	50 KB	1–2
Professional	4 GB	3072 MB	200 KB	2–4

The heap is set to 75% of container memory. The remaining 25% covers JVM metaspace, thread stacks, native memory, and the operating system.

17.7 Bundle ZIP Format

```
bundle.zip
  schemas/                    (optional)
    order.xsd
    invoice.json
  transformations/
    {name}/
      {name}.utlx             (required)
      transform.yaml          (optional)
  engine.yaml                 (optional)
```

17.8 Data Directory Layout

The on-disk state under `/utlxe/data/`. Written by the Admin API, read at startup.

```
/utlxe/data/
  schemas/
    order.xsd
    invoice.json
  transformations/
    invoice-to-ubl/
      invoice-to-ubl.utlx
    order-enrichment/
      order-enrichment.utlx
```

If Azure Files is mounted at `/utlxe/data/`, this directory survives container restarts.

18 Appendix C: UTL-X Quick Reference

A syntax cheat sheet for the most common transformation patterns. For the complete language reference, see the companion book *UTL-X: One Language, All Formats*.

18.1 File Structure

```
%utlx 1.0
input json
output json
---
{ transformation body }
```

18.2 Format Headers

`input json`	JSON input
`input xml`	XML input
`input csv {header: true}`	CSV with header row
`input yaml`	YAML input
`output xml`	XML output
`input json {schema: "f.xsd"}`	With schema validation

18.3 Property Access

`$input.name`	Dot notation
`$input.items[0]`	Array index (0-based)
`$input.@id`	XML attribute
`$input..name`	Recursive descent

18.4 Object Construction

```
{key: value, other: value}
{name: $input.n, ...$input}     // spread operator
```

18.5 Let Bindings

```
let x = expression
```

Multiple bindings separated by newlines or commas.

18.6 Conditionals

```
if (condition) valueA else valueB

match value {
  "A" -> resultA,
  "B" -> resultB,
  _ -> defaultResult
}
```

18.7 Array Operations

`map(arr, (x) -> expr)`	Transform each element
`filter(arr, (x) -> bool)`	Keep matching elements
`reduce(arr, init, (acc, x) -> expr)`	Aggregate to single value
`find(arr, (x) -> bool)`	First matching element
`sortBy(arr, (x) -> key)`	Sort by key function
`groupBy(arr, (x) -> key)`	Group by key function
`flatMap(arr, (x) -> arr)`	Map and flatten
`size(arr)`	Array length
`first(arr)` / `last(arr)`	First or last element

18.8 Pipe Operator

```
$input.items
  | filter(_, (x) -> x.active)
  | map(_, (x) -> x.name)
  | sortBy(_, (x) -> x)
```

The underscore _ is a placeholder for the piped value.

18.9 String Functions

`concat(a, b, c)`	Concatenate
`upperCase(s)` / `lowerCase(s)`	Case conversion
`trim(s)`	Remove whitespace
`replace(s, old, new)`	Replace substring
`contains(s, sub)`	Check substring
`substring(s, start, end)`	Extract substring
`length(s)`	String length
`split(s, delimiter)`	Split to array
`startsWith(s, prefix)` / `endsWith(s, suffix)`	Check prefix or suffix

18.10 Math Functions

`round(n, decimals)`	Round to decimal places
`floor(n)` / `ceil(n)`	Floor or ceiling
`abs(n)`	Absolute value
`min(a, b)` / `max(a, b)`	Minimum or maximum

18.11 Type Functions

`toString(v)` / `toNumber(v)` / `toBoolean(v)`	Type conversion
`isNull(v)`	Null check
`typeOf(v)`	Type name as string

18.12 Date Functions

`now()`	Current timestamp
`formatDate(d, pattern)`	Format a date (e.g., `"yyyy-MM-dd"`)
`parseDate(s, pattern)`	Parse a date string

18.13 Security Functions

`sha256(s)` / `sha512(s)`	Cryptographic hash
`hmacSHA256(s, key)`	HMAC signature
`encryptAES(data, key)` / `decryptAES(data, key)`	AES encryption
`base64Encode(s)` / `base64Decode(s)`	Base64 encoding
`mask(s, start, end, char)`	Mask sensitive data
`env("VAR_NAME")`	Read environment variable (engine only)

18.14 User-Defined Functions

```
function name(param1, param2) {
  expression
}
```

Defined before the main transformation body. Can call standard library functions and other user-defined functions.